CLAMP-RT: Falsification-Robust Fairness Control for Federated Graph Learning via a Two-Server Trimmed Statistic with Attribute-Level Differential Privacy

Quang-Vinh Dang[a,*], Ngoc-Son-An Nguyen[b]

[a] *British University Vietnam, Hung Yen, Vietnam*
[b] *Industrial University of Ho Chi Minh City, Ho Chi Minh City, Vietnam*

Abstract

Fairness control in federated graph learning needs group statistics that clients can falsify. If the server pools range-checked group sums under secure aggregation (a design we call CLAMP), one lying client out of ten raised the demographic-parity gap on Credit from 0.011 to 0.065. We present CLAMP-RT, in which each client secret-shares a single scalar, its soft parity gap on validation nodes with a public group-size floor, between two non-colluding servers. The servers sort and trim the gaps in two-party computation and release a noisy trimmed mean that drives one Lagrange multiplier. We prove that with at most b liars the noise-free trimmed mean stays between two order statistics of the honest gaps, that the release is attribute-level differentially private with a sensitivity fixed by public validation-set sizes, and a bound on the released gap of the returned model (radius 0.19–0.59 on five graphs, vacuous on three small ones). On four graphs with five gap attacks, including an adaptive one, $f \leq b$ liars kept the test gap within 0.009 of the honest run, also with per-round releases, where the controller is active. Combined with update poisoning, $f \leq b$ liars added at most 0.010 to the update attack alone. Beyond the breakdown point the hiding and adaptive attacks raised the gap to 0.10–0.30. A semi-honest CrypTen release round took 1.1–10.5 s. Costs: the honest gap on Credit and Pokec is 0.037–0.061 against 0.018–0.028 for CLAMP, small graphs are noise-dominated, and fairness poisoning through updates is not countered.

*Corresponding author. ORCID: 0000-0002-3877-8024
Email address: vinh.dq4@buv.edu.vn (Quang-Vinh Dang)

Keywords: Federated learning, Graph neural networks, Group fairness, Byzantine robustness, Differential privacy, Secure multi-party computation

1. Introduction

Graph neural networks (GNNs) [1, 2] are used for credit scoring, bail decisions and social recommendation, where the data are split across institutions and the predictions affect people. Federated learning [3] lets such institutions train one model without pooling their graphs. Three requirements then arise together: the model should not discriminate between demographic groups [4, 5]; the federation should learn as little as possible about any person's sensitive attribute [6, 7]; and a few compromised clients should not be able to steer the outcome [8, 9, 10].

Group fairness of the global model is a property of the pooled data, so a federation that wants to enforce it needs some statistic of group behaviour from the clients. That statistic is the weak point. A client that reports it can lie, and fair federated methods that reweight or calibrate using client reports [11, 12, 13] are exposed to such lies by design. A natural design that is compatible with secure aggregation, which we call CLAMP and use as the main baseline, lets the server see only four range-checked sums of per-group prediction mass and node counts, computed under verifiable secure aggregation [14, 15]. Such a design has four weaknesses. (1) One lying client can disable fairness control: in our experiments a single falsifying client out of ten raised the test gap of CLAMP on Credit from 0.011 to 0.065, about the level of training without control. (2) The influence of falsified sums can only be bounded by the liars' weight share, which gives a vacuous bound whenever that share is large. (3) The sensitivity of the sums depends on a cap on group sizes, which must be public, whereas the natural cap depends on the attribute. (4) Secure aggregation is usually simulated without any cryptographic cost, and the Gaussian noise of the analysis is not the discrete mechanism a deployment over a finite ring would use.

The first weakness cannot be fixed inside a sums-only design. A sum of unverifiable but range-valid inputs moves with each input in proportion to that input's weight, and no range check can tell a plausible lie from the truth. Robustness to a minority of liars requires an order statistic, and an order statistic cannot be computed from sums. This paper therefore changes the trust model of the statistics channel. Each client reports one scalar, its own

soft demographic-parity gap on its validation nodes, as additive secret shares to two non-colluding servers, the setting of Prio [15] and ELSA [16]. The servers check ranges, sort the K gaps, discard the b smallest and the b largest, and release the mean of the rest with Gaussian noise. This released scalar drives the same single-multiplier controller as before. We call the protocol CLAMP-RT (robust, trimmed).

None of the ingredients is new on its own: two-server secret sharing with validity checks is the Prio model, trimmed means are classical robust estimators used for Byzantine-robust learning [9], the Gaussian mechanism with Rényi accounting is standard [17], and the multiplier update is projected dual ascent [18]. What we contribute is a specific combination for a specific gap: a one-dimensional, privatised and falsification-robust statistics channel for fairness control in federated GNN training, with guarantees whose scope we state, including where they become vacuous, and with a measured two-party protocol. Concretely:

1. **A falsification-robust statistics channel** (Sections 4 and 5; weaknesses 1 and 2). We prove that with $f \leq b$ liars the noise-free trimmed mean lies between two order statistics of the honest clients' gaps (Theorem 5). Liars therefore cannot move this statistic outside the honest range, and if all honest gaps except at most $b - f$ exceed a level τ' (at $f = b$: every honest gap), the statistic exceeds τ'. The interval is an honest-data quantity rather than a vacuous worst case, but its mean width in our attacks (0.028–0.055) is comparable to the budget $\tau = 0.03$, and the released value adds noise that the bias correction often maps to zero. In experiments with four fixed attack strategies and an adaptive one (all liars report the value that minimises the signed trimmed mean), $f \leq b$ liars left the mean test gap within 0.009 of the honest run, whereas the old sums channel lost control with one liar. With ten releases the honest level on the Pokec graphs is itself close to training without control; we therefore repeated the attacks with per-round releases ($P = 1$), where the multiplier is active, and again found the outcome within 0.006 of the honest level for $f \leq b$. When the same clients also poison their updates, $f \leq b$ liars added at most 0.010 to the gap reached by the update attack alone, and for $f > b$ the combination broke the gap channel. The adaptive attack is a concrete strong single-claim attacker, not a worst case over all strategies; for $f \leq b$ the worst case is the interval of Theorem 5.

2. **An attribute-level privacy guarantee calibrated by a public floor** (weakness 3). Each client's group rates are normalised by $\max(|V_{k,a}|, n_{0,k})$ with $n_{0,k} = \lceil c|V_k| \rceil$, which depends only on the size of the client's validation set. This fixes the sensitivity of the trimmed mean without any attribute-dependent cap, and the experiments use exactly this constant (Theorem 3).
3. **A measured two-party protocol** (weakness 4). We implement the server step in CrypTen [19] and report time, traffic and rounds (Section 7.5). A full release round, including the norm checks on the updates that the gap channel relies on, took 1.1–10.5 s and up to 976 MB for up to 20 clients and models of up to 10^6 parameters; the sort-and-trim part took at most 1.1 s and 107 KB. The training experiments still run the protocol arithmetic in plaintext with continuous Gaussian noise, and we state what a ring implementation with a discrete mechanism would change.
4. **An honest account of the price and the limits** (Sections 7 and 8). With ten noisy releases, the honest gap of CLAMP-RT is higher than that of CLAMP on Credit and the Pokec graphs (0.037–0.061 against 0.018–0.028). Noise-free runs show that this is not only the privacy noise: on the Pokec graphs per-round noise-free releases reach 0.017–0.019, but on Credit the noise-free trimmed channel stays at 0.035–0.048, which points to the per-client proxy. The bound on the returned model (Theorem 6) has radius 0.19 on the Pokec graphs, 0.40 on Credit and 0.59 on Bail, and is vacuous on the three small graphs, where the noise exceeds any plausible gap and the controller acts on noise. Robustness ends at the breakdown point $f = b + 1$. The trimmed channel does nothing against fairness poisoning through model updates, which the earlier protocol did limit at $P = 1$.

The paper is organised as follows. Section 2 positions the work. Section 3 states the setting and threat model, Section 4 the protocol and Section 5 the guarantees, with full proofs in the supplementary material. Sections 6 and 7 report the experiments, and Section 8 the limitations.

2. Related work

Fair graph neural networks. Centralised fair GNNs remove sensitive information from representations or rebalance training: FairGNN [4] uses an adversary

to estimate and remove the attribute, NIFTY [5] enforces counterfactual stability, EDITS [20] debiases attributes and structure, and FairSIN [21] neutralises sensitive information through added features; re-balancing [22] and attribute-agnostic methods [23] pursue the same goal. All of them assume that one party holds the graph and the attribute. We use FairGNN and FairSIN as baselines, trained with federated averaging.

Fairness in federated learning. FairFed [11] reweights clients by the gap between local and global fairness, q-FFL [12] reweights by local loss, and FedFACT [13] calibrates a trained model to a group-fairness target. PUFFLE [24] and FedFDP [25] combine fairness with differential privacy; FedFDP targets fairness of performance across clients. For graphs, F^2GNN [26] combines structure-aware local fairness with fairness-weighted aggregation, FairGFL [27] handles overlapping subgraphs under privacy, and FedGraph-Fair [28] uses personalisation and distributionally robust optimisation. Bendoukha et al. [29] compute FairFed's weights under homomorphic encryption, which hides the reports but still trusts their values. These methods read per-client fairness or loss values, in the clear or encrypted, and assume that the values are honest. Our earlier conference work [30] addressed privacy and fairness for fraud detection with client-side mechanisms and did not consider Byzantine clients.

Constrained learning. Enforcing fairness as a constraint through a Lagrangian is standard in centralised learning [18]. Our dual update has one multiplier. The questions we study are about its input: the statistic that drives it is privatised, computed under secure computation and partly controlled by an adversary.

Byzantine robustness and secure aggregation. Robust aggregation rules (Krum [8], the coordinate-wise median and trimmed mean [9], Bulyan [31]) and trust-scored rules such as FLTrust [32] inspect individual updates, and attacks that stay within the spread of honest updates evade them [10, 33]. Secure aggregation [14] hides individual updates and thereby rules these defences out. Several lines of work restore some robustness under encryption. Prio [15] computes aggregate statistics between non-colluding servers with zero-knowledge validity proofs on secret-shared inputs; RoFL [34] enforces norm bounds on encrypted updates with a single server; BREA [35] and ELSA [16] provide Byzantine-resilient or malicious-secure aggregation of model updates. These systems protect the *model update* channel. None of them addresses the statistic that a fairness controller needs, and none

combines its robustness with an attribute-level privacy guarantee for that statistic. Distributed differential privacy with secure aggregation [36] provides the discrete noise mechanism that a deployment over a finite ring would use.

Robust and private statistics, and attacks on fairness. Trimmed means are classical robust location estimators, and the coordinate-wise trimmed mean of Yin et al. [9] gives statistical rates for Byzantine-robust learning. Dwork and Lei [37] connected differential privacy and robust statistics, observing that estimators with bounded sensitivity to single records are natural candidates for privatisation; our trimmed mean of floored per-client gaps is a simple instance, with sensitivity bounded by Lemma 1. Fairness itself can be attacked: poisoning attacks that target fairness rather than accuracy were studied for centralised learning [38, 39]. Our update-channel attack of the same kind (fairness poisoning) is evaluated in Section 7.4, and the statistics attacks of Section 7.2 target the fairness estimate instead of the training data.

Positioning. CLAMP-RT reuses the two-server model of Prio and ELSA and the trimmed mean of the robust-aggregation literature, but applies them to a different object: a single scalar per client that summarises its group-fairness gap, sorted and trimmed inside two-party computation, privatised at the level of one validation node's sensitive attribute, and fed to a fairness controller. Because the object is one-dimensional, sorting it in secure computation is cheap (Section 7.5), whereas robust aggregation of d-dimensional updates in secure computation is costly. The update channel of CLAMP-RT is still a norm-checked sum; we do not claim robustness beyond the per-round influence bound of clipping for that channel.

3. Problem setting and threat model

Federated node classification. K clients each hold a subgraph $G_k = (V_k, E_k)$ with node features, binary labels y_v and a binary sensitive attribute $s_v \in \{0, 1\}$. Each client splits its labelled nodes into training, validation and test sets; we write $\mathcal{V}_k$ for its validation nodes and $\mathcal{V}_{k,a} = \{v \in \mathcal{V}_k : s_v = a\}$. A GNN f_θ maps each node to a probability $p_\theta(v) \in (0, 1)$. The attribute is removed from the features, so the backbone never reads it. The server coordinates T rounds of training of one global model.

Fairness target. We evaluate the soft demographic-parity gap of the global model on the pooled test nodes,

$$\mathrm{DPD}(\theta) = \big|r_1(\theta) - r_0(\theta)\big|, \qquad r_a(\theta) = \frac{\sum_k \sum_{v \in V_k^{\mathrm{test}}, s_v = a} p_\theta(v)}{\sum_k \big|\{v \in V_k^{\mathrm{test}} : s_v = a\}\big|}, \tag{1}$$

and the federation wants a model with DPD $\leq \tau$ at the smallest cost in utility. The controller of CLAMP-RT does not see the pooled gap. It sees a trimmed mean of the clients' own validation gaps (Section 4), which is a robust proxy: under non-IID partitions a client's gap can differ in sign and size from the pooled one, and the trimmed mean of per-client gaps can differ from the pooled gap. We accept this mismatch as the cost of an order statistic, and we measure the result on the pooled test gap (1).

Servers. There are two aggregation servers, A and B, which do not collude and follow the protocol (semi-honest); each may try to learn as much as possible from what it sees. This is the trust model of Prio [15]. The servers must not learn any individual client's update or gap, and what they release must not reveal any validation node's sensitive attribute beyond a differential-privacy budget. Our cost measurements use a semi-honest two-party protocol; protection against a malicious server would need a maliciously secure protocol and is not covered.

Byzantine clients. Up to f of the K clients are Byzantine. They know the protocol, the public constants and the current global state, may coordinate, and may submit any model update and any gap. Submissions that fail the servers' validity checks (norm of the update, range of the gap) are discarded. We distinguish *statistics attacks*, in which liars report false gaps to make the controller believe the model is fairer (or less fair) than it is (we allow them to be omniscient: they know every honest gap of the round and the public b, and may choose their claims adaptively in each release), and *update attacks*, in which they manipulate their model updates, including *fairness poisoning*, where they train to widen the gap while still fitting the labels.

Public constants. The number of clients K, the trimming level b, the floor fraction c, the sizes $|\mathcal{V}_k|$ of the clients' validation sets, the norm bound C, the budget τ, the step η, the release period P and the noise level σ_d are public. The validation-set sizes do not depend on any sensitive attribute; the sizes of the groups $|\mathcal{V}_{k,a}|$ do, and they are never revealed.

4. The CLAMP-RT protocol

CLAMP-RT keeps the training loop of CLAMP: clipped updates are summed, and one broadcast Lagrange multiplier λ with one broadcast direction $\omega \in \{-1, 0, 1\}$ adds a signed fairness penalty to every client's loss. It replaces the statistics channel. Algorithm 1 lists one round.

4.1. Client side

Local gap. In a release round the client evaluates the received model θ^t on its validation nodes with a forward pass that does not read s, and computes

$$m_{k,a} = \frac{\sum_{v \in \mathcal{V}_{k,a}} p_{\theta^t}(v)}{\max\Big(|\mathcal{V}_{k,a}|,\, n_{0,k}\Big)}, \qquad d_k = \mathrm{clip}\Big(m_{k,1} - m_{k,0},\, -1,\, 1\Big), \tag{2}$$
$$n_{0,k} = \max\Big(1, \lceil c\,|\mathcal{V}_k| \rceil\Big).$$

The floor $n_{0,k}$ depends only on the public size of the client's validation set. For a group with at least $n_{0,k}$ nodes, $m_{k,a}$ is the group's mean prediction; a smaller group is shrunk toward zero, which biases d_k but bounds its sensitivity (Lemma 2). We use $c = 0.05$. The clip never binds for an honest client, since $m_{k,a} \in [0, 1]$.

Lagrangian local objective. Starting from θ^t, the client runs E local epochs on its training nodes with

$$\mathcal{L}_k(\theta) = \mathrm{BCE}_w\Big(p_\theta, y\Big) + \lambda^t \omega^t \Big(\bar{p}_{k,1}(\theta) - \bar{p}_{k,0}(\theta)\Big), \tag{3}$$

where BCE_w is class-weighted cross-entropy and $\bar{p}_{k,a}$ is the mean prediction over the client's training nodes in group a. It then clips its pseudo-gradient, $\Delta_k \leftarrow (\theta^t - \theta_k)\min(1, C/\|\theta^t - \theta_k\|_2)$.

Submission. The client splits Δ_k and, in a release round, d_k into additive secret shares over the ring $\mathbb{Z}_{2^{64}}$ with fixed-point encoding, and sends one share to each server. Neither server alone learns anything about Δ_k or d_k.

4.2. Server side

The two servers run the following steps jointly in two-party computation.

1. *Validity checks.* Client k is accepted if $\|\Delta_k\|_2 \le C$ and, in a release round, $-1 \le d_k \le 1$. A rejected client is removed from the update sum, and its gap is replaced by 0, so that the sort always has K entries. Honest clients are always accepted.

2. *Update channel.* The servers open $U = \sum_{k \in A} \Delta_k$ for the accepted set A and set $\theta^{t+1} = \theta^t - U/|A|$.
3. *Gap channel (release rounds).* The servers sort the K entries $\tilde{d}_k$ (d_k if accepted, else 0), keep the middle $K - 2b$ values and compute

$$\mathrm{TM}_b(\tilde{d}) = \frac{1}{K - 2b} \sum_{i=b+1}^{K-b} \tilde{d}_{(i)}. \tag{4}$$

   Each server adds its own noise $\xi_A, \xi_B \sim \mathcal{N}(0, \sigma_d^2)$ to its share, and the opened value is $\tilde{g} = \mathrm{TM}_b(\tilde{d}) + \xi_A + \xi_B$. We use $b = \max(1, \lfloor 0.2K \rfloor)$, i.e. $b = 1$ for $K = 5$ and $b = 2$ for $K = 10$.

The servers learn U, $\tilde{g}$ and the number $|A|$ of accepted clients, which is needed to average the update sum. In our implementation the validity bits stay secret-shared, so the identities of rejected clients are not opened; a deployment that informs rejected clients, or whose network layer shows which clients sent nothing, reveals them. The servers do not learn the individual gaps, which clients were trimmed, or any group size. The model θ^{t+1} is broadcast and therefore public.

Controller. From the released $\tilde{g}$ the server forms a magnitude, updates the multiplier by projected dual ascent and sets the direction with a dead zone:

$$\widehat{D} = \sqrt{\max\bigl(0,\ \tilde{g}^{\,2} - 2\sigma_d^2\bigr)}, \qquad \lambda^{t+1} = \max\bigl(0, \lambda^t + \eta(\widehat{D} - \tau)\bigr),$$
$$\omega^{t+1} = \begin{cases} \mathrm{sign}(\tilde{g}) & \text{if } \widehat{D} > \tau/2, \\ 0 & \text{otherwise.} \end{cases} \tag{5}$$

The first expression is a bias correction. Because $\mathbb{E}[\tilde{g}^{\,2}] = \mathrm{TM}_b^2 + 2\sigma_d^2$, the plain magnitude $|\tilde{g}|$ overestimates $|\mathrm{TM}_b|$ when σ_d is comparable to the gap; in our first runs this drove the multiplier up on the small graphs and the controller destroyed the model there (Table 2, row "no bias correction"). We use the correction as a heuristic: $\widehat{D}$ is not an unbiased estimator of $|\mathrm{TM}_b|$, and we do not claim any optimality for it. It is a function of the released value and the public σ_d, so it costs no privacy. Outside release rounds, λ and ω keep their last values.

Output rule. The signed penalty makes the gap oscillate around the budget, so the last iterate lands at an arbitrary point of the oscillation. As in CLAMP,

Algorithm 1 One round t of CLAMP-RT

Require: $\theta^t, \lambda^t, \omega^t$; public $C, \tau, \eta, b, c, |\mathcal{V}_k|, \sigma_d, P, W$

Servers broadcast $(\theta^t, \lambda^t, \omega^t)$
for each client k **in parallel do**
  if $t \bmod P = 0$ **then**
    compute d_k with θ^t by Eq. (2)
  end if
  $\theta_k \leftarrow E$ local epochs on $\mathcal{L}_k$, Eq. (3); $\Delta_k \leftarrow (\theta^t - \theta_k)\min(1, C/\|\theta^t - \theta_k\|_2)$
  secret-share Δ_k (and d_k) between servers A and B
end for
In 2PC: validity checks; open $U = \sum_{k \in A} \Delta_k$; $\theta^{t+1} \leftarrow \theta^t - U/|A|$
if $t \bmod P = 0$ **then**
  In 2PC: sort $\tilde{d}$, trim b per side, add $\xi_A + \xi_B$, open $\tilde{g}$
  update $\widehat{D}, \lambda^{t+1}, \omega^{t+1}$ by Eq. (5)
  if $t + 1 \geq W$ **and** $\widehat{D} \leq \tau$ **then** store $\hat{\theta} \leftarrow \theta^t$
else
  $(\lambda^{t+1}, \omega^{t+1}) \leftarrow (\lambda^t, \omega^t)$
end if
after round T**:** return $\hat{\theta}$, or θ^T if nothing was stored

the server stores the model θ^t of each release round with $t + 1 \geq W$ (rounds counted from $t = 0$) and $\widehat{D}^t \leq \tau$, and returns the most recently stored one, or the last model if none was stored. With the bias correction, $\widehat{D} \leq \tau$ is equivalent to $|\tilde{g}| \leq \sqrt{\tau^2 + 2\sigma_d^2}$, a looser test than $|\tilde{g}| \leq \tau$; Section 7 shows what this does on the small graphs. We use $W = \lceil 0.2T \rceil$.

Release period and noise. Statistics are released every P rounds, $R = \lceil T/P \rceil$ times in total. The noise level is $\sigma_d = \Delta_{\mathrm{TM}}\, z$, where $\Delta_{\mathrm{TM}} = 2/(n_{0,\min}(K - 2b))$ with $n_{0,\min} = \min_k n_{0,k}$ is the sensitivity of Theorem 3 and z is the smallest noise multiplier for which R Gaussian releases are (ε, δ)-differentially private under Rényi accounting. We use $P = 10$, $\varepsilon = 8$ and $\delta = 10^{-5}$ on all datasets, which gives $R = 10$ and $z = 2.02$.

What changed relative to CLAMP. CLAMP released four noisy sums (S_0, S_1, N_0, N_1) and formed the pooled gap $S_1/N_1 - S_0/N_0$. Its sensitivity was set by a cap on group sizes, and its only defence against liars was the range

check $0 \le S_a \le N_a \le n_{\text{cap}}$. CLAMP-RT releases one noisy order-statistic average of per-client gaps. The price is that the controller targets a proxy of the pooled gap and that the noise is added to a statistic whose sensitivity scales with $1/(n_{0,\min}(K-2b))$ rather than with the inverse of the pooled group size.

5. Guarantees

This section states four results, one per channel or property, together with their scope. Full proofs are in Section S1 of the supplementary material; we give the key steps here. None of the results is a convergence theorem for GNN training. We write $x_{(1)} \le \cdots \le x_{(K)}$ for the sorted entries of $x \in \mathbb{R}^K$.

5.1. Two facts about order statistics

Lemma 1 (Order statistics under a change of one entry)**.** *Let $x, y \in \mathbb{R}^K$ differ only in entry j. Then $x_{(i)} - y_{(i)}$ has the same sign for every i (or is zero), and $\sum_{i=1}^K |x_{(i)} - y_{(i)}| = |x_j - y_j|$. Consequently $|\mathrm{TM}_b(x) - \mathrm{TM}_b(y)| \le |x_j - y_j|/(K-2b)$. Moreover, for any $x, y \in \mathbb{R}^K$, $\max_i |x_{(i)} - y_{(i)}| \le \max_i |x_i - y_i|$, so TM_b is 1-Lipschitz in ℓ_∞.*

Proof sketch. Each order statistic $x_{(i)} = \max_{|S|=K-i+1} \min_{l \in S} x_l$ is non-decreasing in every entry. If $y_j \ge x_j$, then $y_{(i)} \ge x_{(i)}$ for all i, and summing gives $\sum_i (y_{(i)} - x_{(i)}) = \sum_i y_i - \sum_i x_i = y_j - x_j$. Restricting the sum of non-negative terms to the kept indices and dividing by $K-2b$ gives the bound on TM_b. For the second claim, $y \le x + \epsilon \mathbf{1}$ entrywise implies $y_{(i)} \le x_{(i)} + \epsilon$ by monotonicity and translation equivariance, and symmetrically. □

Lemma 2 (Sensitivity of a client's gap)**.** *Fix the predictions $p_\theta(v)$ of client k's validation nodes. Changing the sensitive attribute of one validation node changes d_k in (2) by at most $2/n_{0,k}$.*

Proof sketch. The node, with prediction $p \in [0,1]$, leaves one group and joins the other. For the group it joins, with N nodes and prediction mass $S \in [0, N]$ before the change, either $N+1 \le n_0$, so that both denominators equal n_0 and the rate changes by p/n_0, or $N \ge n_0$, so that the change is $|(S+p)/(N+1) - S/N| = |pN - S|/(N(N+1)) \le 1/(N+1) \le 1/n_0$. The group it leaves is the same computation in reverse. Hence each of $m_{k,0}, m_{k,1}$ moves by at most $1/n_{0,k}$, their difference by at most $2/n_{0,k}$, and the clip is 1-Lipschitz. □

5.2. Attribute-level differential privacy of the gap channel

Definition 1 (Attribute-neighbouring datasets)**.** Two federated datasets are *neighbouring* if they are identical except for the sensitive attribute s_v of a single validation node v of one honest client.

Theorem 3 (Privacy of the released gaps)**.** *Let* $\Delta_{\mathrm{TM}} = 2/(n_{0,\min}(K-2b))$, $\sigma_d = z\Delta_{\mathrm{TM}}$ *and* $R = \lceil T/P \rceil$. *Fix all training data, the initialisation and the randomness of local training, and let the Byzantine clients' submissions be arbitrary functions of public information and of their own data. If at least one server is honest, the sequence* $(\tilde{g}^{(1)}, \ldots, \tilde{g}^{(R)})$ *of released gaps satisfies* $(\alpha, R\alpha/(2z^2))$*-Rényi differential privacy [17] for every* $\alpha > 1$ *with respect to attribute-neighbouring datasets, and therefore* (ε, δ)*-differential privacy with* $\varepsilon = \min_{\alpha>1}\Big[R\alpha/(2z^2) + \log\frac{\alpha-1}{\alpha} - \frac{\log\delta+\log\alpha}{\alpha-1}\Big]$ *[40]. The same holds for* $\widehat{D}$, λ, ω, *the models and the returned model, which are post-processing of the releases and of the update channel.*

Proof sketch. Consider one release with a fixed history of earlier releases. The changed node's attribute is not read by the backbone or by local training (which uses training nodes only), so every model, every update and the acceptance decision of every client are identical in the two datasets; only d_k changes, by at most $2/n_{0,k} \le 2/n_{0,\min}$ (Lemma 2). By Lemma 1 the trimmed mean changes by at most Δ_{TM}. The honest server's noise $\mathcal{N}(0, \sigma_d^2)$ alone makes the release a Gaussian mechanism with sensitivity Δ_{TM}, which is $(\alpha, \alpha\Delta_{\mathrm{TM}}^2/(2\sigma_d^2)) = (\alpha, \alpha/(2z^2))$-RDP; the other server's noise can only help, and if that server is the adversary it knows its own noise, which is why the analysis uses σ_d and not $\sqrt{2}\sigma_d$. Under the two-party protocol, a server's view is simulatable from the opened values. Adaptive composition over R releases and the conversion of [40] give the claim. □

Remark 1 (Public versus attribute-dependent caps). The only constant that fixes the sensitivity is $n_{0,\min}$, which is computed from the validation-set sizes $|\mathcal{V}_k|$. These sizes do not depend on any attribute and are public in our model; the group sizes $|\mathcal{V}_{k,a}|$ are never used outside the secure computation. The experiments compute $n_{0,k}$ in exactly this way, so the theorem applies to every run reported here, unlike an attribute-dependent cap. The price is the shrinkage of groups smaller than $n_{0,k}$; in our honest runs this happened for no client-group except four on NBA (over ten seeds).

Remark 2 (Scope). The guarantee covers the attributes of validation nodes, which are a quarter of the labelled nodes in our splits. Training-node attributes

enter the local penalty (3) and therefore the update sum U and the broadcast models. Secure computation hides each client's individual update, but U and the models are revealed to the servers (the models to every client), and they are not differentially private. The test nodes are not used. At $\varepsilon = 8$ the guarantee is weak as a bound on an attacker's success, which Section 7.6 quantifies. The analysis assumes continuous Gaussian noise added to a real-valued trimmed mean. A deployment over $\mathbb{Z}_{2^{64}}$ must add discrete noise, for example the discrete Gaussian [40, 36], to a fixed-point value; the sensitivity then includes the rounding error and the accounting changes accordingly. We have not implemented or analysed this version.

5.3. Per-round influence of Byzantine updates

Theorem 4 (Update influence)**.** *Fix the state at round t. Let A be the accepted set and $B \subseteq A$ the accepted Byzantine clients, $|B| = f' \le f$, and $H = A \setminus B$. If all honest clients are accepted, then $\left\| U/|A| - \frac{1}{|H|} \sum_{k \in H} \Delta_k \right\|_2 \le 2f'C/|A| \le 2fC/(K-f)$.*

This is the bound of any clipped mean, unchanged from CLAMP (proof in Section S1). It is per round, against a same-state counterfactual, and not small: at $f = 4$ of $K = 10$ it is $1.33C$. The trimmed gap channel does not improve it.

5.4. Falsified gaps

Theorem 5 (Falsified gaps cannot leave the honest range)**.** *Suppose $f \le b$ clients are Byzantine and all honest clients are accepted. Let $h_{(1)} \le \cdots \le h_{(K-f)}$ be the sorted honest gaps. Whatever values in $[-1, 1]$ the Byzantine clients submit (a value outside fails the range check and enters as 0), every kept order statistic satisfies $h_{(i-f)} \le \tilde{d}_{(i)} \le h_{(i)}$ for $b+1 \le i \le K-b$, and therefore*

$$h_{(b-f+1)} \;\le\; \frac{1}{K-2b} \sum_{i=b+1}^{K-b} h_{(i-f)} \;\le\; \mathrm{TM}_b(\tilde{d}) \;\le\; \frac{1}{K-2b} \sum_{i=b+1}^{K-b} h_{(i)} \;\le\; h_{(K-b)}. \tag{6}$$

Proof. Fix i with $b+1 \le i \le K-b$. At least i entries are $\le \tilde{d}_{(i)}$, and at most f of them are Byzantine, so at least $i - f \ge b - f + 1 \ge 1$ honest gaps are $\le \tilde{d}_{(i)}$, i.e. $h_{(i-f)} \le \tilde{d}_{(i)}$. At least $K - i + 1$ entries are $\ge \tilde{d}_{(i)}$, at most f of them Byzantine, so at least $K - f - i + 1 \ge b - f + 1 \ge 1$ honest gaps

are $\geq \tilde{d}_{(i)}$, i.e. $\tilde{d}_{(i)} \leq h_{(i)}$ (note $i \leq K - b \leq K - f$). Average over the kept indices and use monotonicity of $h_{(\cdot)}$. □

The bound is an interval between honest values. In this qualitative sense it is not vacuous, unlike a weight-share bound for summed statistics, which allows the estimate to move anywhere in $[0, 1]$ once the liars' share is large; quantitatively, as (iii) shows, it is not informative at the scale of the budget. All statements concern the noise-free statistic $\mathrm{TM}_b(\tilde{d})$. Three consequences follow.

(i) Liars cannot place the estimate outside $[h_{(b-f+1)}, h_{(K-b)}]$.

(ii) If all honest gaps except at most $b - f$ exceed τ', then $\mathrm{TM}_b(\tilde{d}) > \tau'$, whatever the liars send. At $f = b$ this requires *every* honest gap to exceed τ', a strong condition under non-IID partitions. With noise, on the event $|\xi_A + \xi_B| \leq r$ the released $|\tilde{g}|$ exceeds $\tau' - r$, but this does not guarantee that the controller acts: the bias-corrected magnitude $\widehat{D}$ is zero whenever $|\tilde{g}| \leq \sqrt{2}\sigma_d$, which is 0.065–0.100 on Credit, above the budget. In the honest runs on Credit, Pokec-z and Pokec-n only 42–59% of releases had $\widehat{D} > 0$ (Table 3). The statement therefore says that liars cannot *hide* a gap that all honest clients see, not that the controller will respond to it.

(iii) Let $\mathrm{TM}_{b-f}(h)$ be the trimmed mean of the $K - f$ honest gaps with $b - f$ values removed per side; it averages $h_{(b-f+1)}, \ldots, h_{(K-b)}$ and so lies in the same interval as $\mathrm{TM}_b(\tilde{d})$. Hence $|\mathrm{TM}_b(\tilde{d}) - \mathrm{TM}_{b-f}(h)| \leq h_{(K-b)} - h_{(b-f+1)}$: the effect of the liars relative to this honest-only comparator is at most the honest inner spread, which the server cannot observe but our logs record. In the attacked release rounds with $f \leq b$ its mean was 0.028–0.055 and its maximum 0.090 (Table S3), comparable to the budget $\tau = 0.03$. The bound is therefore informative but not tight compared with τ.

The theorem, not any experiment, is the statement about the worst case over adaptive strategies for $f \leq b$; Section 7.3 evaluates one concrete adaptive attacker. For $f > b$ at least one Byzantine value can be kept, and each kept value can move the trimmed mean by up to $2/(K - 2b)$ (e.g. 2/3 for $K = 5$, $b = 1$): this is the breakdown point, and Section 7.2 shows that it is reached in practice. The theorem says nothing about the update channel; a Byzantine client that reports an honest gap but poisons its update is covered only by Theorem 4.

5.5. The released gap of the returned model

Theorem 6 (Returned model)**.** *Let* $r_R = 2\sigma_d\sqrt{\ln(2R/\beta)}$ *and* $\tau_{\text{eff}} = \sqrt{\tau^2 + 2\sigma_d^2}$. *With probability at least* $1-\beta$ *over the noise of the* R *releases, simultaneously for all release rounds,* $|\tilde{g} - \mathrm{TM}_b(\tilde{d})| \le r_R$. *On this event, if the output rule returns a stored model* $\hat{\theta}$ *(no fallback), then* $|\mathrm{TM}_b(\tilde{d}(\hat{\theta}))| \le \tau_{\text{eff}} + r_R$, *and if* $f \le b$ *the honest gaps of* $\hat{\theta}$ *satisfy* $h_{(b-f+1)}(\hat{\theta}) \le \tau_{\text{eff}} + r_R$ *and* $h_{(K-b)}(\hat{\theta}) \ge -(\tau_{\text{eff}} + r_R)$.

Proof sketch. Since $\xi_A + \xi_B \sim \mathcal{N}(0, 2\sigma_d^2)$, we have $\Pr[|\xi_A + \xi_B| > r_R] \le \beta/R$; a union bound over the R releases gives the event. The stored model satisfies $\widehat{D} \le \tau$, i.e. $|\tilde{g}| \le \tau_{\text{eff}}$, hence $|\mathrm{TM}_b| \le \tau_{\text{eff}} + r_R$ on the event. The last claim is Theorem 5 applied to $\hat{\theta}$. □

Compared with the analogous statement for summed statistics, there is one noisy scalar per release instead of four, and the bound does not depend on unobservable honest group masses. It is, however, a statement about validation gaps of the per-client statistic, it is conditional on the rule not falling back to the last model, and it is loose. The fallback is observable; it did not occur in any honest run (E1, E5), occurred in 7 of 126 update-attack runs (E3), never occurred in the falsification runs with $f \le b$, and occurred in 35 of the 96 falsification runs beyond the breakdown point, where the multiplier ran away (Section 7.2). Table 3 evaluates the radius $\tau_{\text{eff}} + r_R$ at the noise levels of our runs ($\beta = 0.05$): 0.19 on the Pokec graphs, 0.40 on Credit and 0.59 on Bail, and above 1, hence vacuous, on German, NBA and Facebook.

Proposition 7 (From validation to population, under an independence assumption)**.** *Suppose that, conditionally on the group sizes* $n_{k,a} = |\mathcal{V}_{k,a}| \ge n_{0,k}$, *client* k*'s validation nodes of group* a *are i.i.d. draws from a distribution with mean prediction* $\mu_{k,a}(\hat{\theta})$ *under the returned model, and that* $\hat{\theta}$ *is independent of these draws. Then with probability at least* $1-\beta'$, $|m_{k,a} - \mu_{k,a}(\hat{\theta})| \le \sqrt{\ln(4K/\beta')/(2n_{k,a})} =: e_{k,a}$ *for all* k, a, *and the trimmed mean of the population gaps* $\mu_{k,1} - \mu_{k,0}$ *is within* $2\max_{k,a} e_{k,a}$ *of* $\mathrm{TM}_b(d(\hat{\theta}))$.

The proof is Hoeffding's inequality with a union bound over the $2K$ client-groups and the ℓ_∞ Lipschitz property of Lemma 1. The independence assumption does *not* hold exactly for CLAMP-RT: the returned model is chosen using the releases, and the training trajectory depends on earlier

Table 1: Datasets after preprocessing. "Kept" is the fraction of edges inside a client after the $K = 5$ partition (seed 0). $P(y{=}1)$ is over labelled nodes.

Dataset	Nodes	Edges	Features	$P(y{=}1)$	$P(s{=}1)$	Kept
German	1,000	21,742	26	0.700	0.310	0.21
Bail	18,876	311,870	16	0.376	0.506	0.38
Credit	30,000	1,421,858	12	0.779	0.090	0.34
Pokec-z	67,796	617,958	276	0.081	0.352	0.29
Pokec-n	66,569	517,047	265	0.064	0.289	0.29
AMiner-S	39,424	52,460	256	0.197	0.403	0.45
NBA	403	10,621	95	0.508	0.266	0.33
Facebook	1,045	26,749	573	0.787	0.342	0.31

releases, which depend on the validation nodes. A union bound over the R candidate models does not repair this, because the candidates themselves depend on the validation data. We therefore present Proposition 7 as a description of the sampling error that the validation statistic carries, not as a guarantee of the protocol, and we evaluate the gap on held-out test nodes.

6. Experimental setup

Datasets. We use eight public benchmarks (Table 1), all with a genuine binary sensitive attribute. *German* (credit risk; gender) and *Credit* (credit default; age) are similarity graphs over individuals, and *Bail* (bail decisions; race) links defendants with similar records [5]. *Pokec-z* and *Pokec-n* (region) are the regional subsets of the Pokec social network used in [4], and *NBA* (nationality) links basketball players [4]. *Facebook* (gender) is ego network 107 of the Facebook social circles data [41], and *AMiner-S* is a co-authorship network whose attribute is the continent of the author's affiliation (most frequent continent against the rest), taken from PyGDebias [42], with features reduced to 256 dimensions by truncated SVD. The sensitive attribute is removed from the features. Each run draws a stratified 50/25/25 train/validation/test split and partitions the nodes over K clients with a Dirichlet($\alpha = 0.3$) split over labels; each client keeps the subgraph induced by its nodes. The split, the partition and the initialisation depend on the seed.

Protocol constants. All methods train for $T = 100$ rounds with $E = 3$ full-batch local epochs, AdamW (learning rate 0.01, weight decay 10^{-5}), two-layer backbones with 64 hidden units (GAT with 4 heads) and dropout 0.3.

CLAMP-RT uses $C = 1$, $\eta = 0.5$, $\tau = 0.03$, $W = 20$, $b = \max(1, \lfloor 0.2K \rfloor)$, $c = 0.05$, $P = 10$ and $\varepsilon = 8$, $\delta = 10^{-5}$ for Theorem 3, on every dataset. The resulting $n_{0,\min}$ and σ_d are listed in Table 3. The constants C, η, τ, W are those of CLAMP. The values of c and P were chosen in pilot runs on Credit, Pokec-z and Bail (the design started from $c = 0.1$; we also tried and discarded exponential smoothing of the released gap). The bias correction was added after the first full honest runs, which are reported as the row "no bias correction" in Table 2, and was piloted on six datasets including the three small graphs. The pilots used seeds 0–2, which are among the reported seeds, so the reported results are not fully independent of these choices. All pilots, including the discarded variants (smoothing, a step scaled by the release period, a tighter controller target), are listed in Table S5 of the supplementary material. Tables 12 and 13 report the other values of P, b and c. No constant was tuned per dataset. CLAMP-RT uses a GAT backbone and CLAMP-RT-GCN a GCN backbone.

Simulation of the secure computation. The training experiments execute the server-side arithmetic of Section 4 in plaintext, in 64-bit floating point, with continuous Gaussian noise: validity checks, sort, trimming, noise and the masked update sum are computed exactly as specified, and the server code reads only the opened values. The cost of running the same steps in two-party computation is measured separately (Section 7.5). The fixed-point error of that implementation is below 1.5×10^{-5} on the trimmed mean, far below the noise, but the discrete noise a ring deployment would need is not part of either experiment.

Baselines. All baselines use the same splits, partitions and seeds as CLAMP-RT: FedAvg [3] with GAT; the fair GNNs FairGNN [4] and FairSIN [21] trained with FedAvg; FairFed [11] and FedFACT [13]; the federated graph methods F^{2}GNN [26], FairGFL [27] and FedGraph-Fair [28]; and the private methods PUFFLE [24] and FedFDP [25]. FairFed, F^{2}GNN and FedFACT read per-client fairness quantities or group rates in the clear. PUFFLE and FedFDP account privacy per local step for gradients at $\varepsilon = 8$; their privacy unit differs from ours and the two values of ε are not comparable. ClipMean is clipped aggregation without a controller. The most important comparison is with CLAMP, the sums-channel protocol: it differs from CLAMP-RT only in the statistics channel, except that it released statistics every round ($P = 1$) on Bail, Credit, Pokec-z, Pokec-n and AMiner-S and every ten rounds on the

three small graphs. For CLAMP we apply the same output-rule warm-up as for CLAMP-RT.

Metrics and statistics. We report test AUC and the soft demographic-parity gap DPD of Eq. (1) on pooled test nodes, as mean ± standard deviation over seeds. The honest comparison (E1) uses ten seeds; attack studies, the privacy sweep and the ablations use three, so differences below about 0.01 in DPD are within seed noise there. For CLAMP-RT against CLAMP we report two-sided Wilcoxon signed-rank tests paired by seed.

Experiments. E1 (honest): eight datasets, $K = 5$. *E3* (update attacks): $K = 10$, $f \in \{1, \dots, 4\}$ Byzantine clients performing sign flipping, scaling, ALIE [10], inner-product manipulation [33] or fairness poisoning, on Bail and Pokec-z; Byzantine clients report their true gaps. *E4* (falsified gaps): Bail, Credit, Pokec-z and Pokec-n with $K = 5$ ($f \in \{1, 2\}$, $b = 1$) and $K = 10$ ($f \in \{1, 2, 3\}$, $b = 2$), so that each setting includes one value of f beyond the breakdown point. Liars train honestly and replace their gap by one of four omniscient claims within $[-1, 1]$, computed from the honest gaps of the round with mean g^H: *hide* reports $-\text{sign}(g^H)$, *inflate* reports $+\text{sign}(g^H)$, *inlier* reports the honest value closest to zero on the hiding side (the strongest claim that stays inside the honest range), and *zero* reports 0. An adaptive fifth attack, *opt*, lets all liars report the common value, chosen from a grid of step 0.01 on $[-1, 1]$ together with the honest values, that minimises $\text{sign}(g^H)\,\text{TM}_b$ of the resulting vector; it is the myopically optimal single hiding claim against the trimmed mean with public b (E7, same settings as E4). E8 repeats the hide, inflate, inlier and opt attacks with per-round releases ($P = 1$, $R = 100$) for $K = 10$, $f \in \{1, 2, 3\}$ on Credit, Pokec-z and Pokec-n, with an honest reference. For CLAMP the liars use a *swap* attack on the four sums with the public cap. *E5*: $\varepsilon \in \{1, 2, 4, 8\}$ and $P \in \{1, 5, 10\}$ on Credit, Pokec-z and Pokec-n. *E6*: ablations of b, c and the output rule on Credit and Pokec-z.

7. Results

7.1. Honest runs

Table 2 reports AUC and DPD of all methods without attack, and Table 3 the state of the gap channel and the paired comparison with CLAMP.

Table 2: Honest runs ($K = 5$): test AUC and soft DPD (lower is fairer), mean±sd over 10 seeds. Baselines and CLAMP (sums channel) use the same seeds and splits, with the output-rule warm-up applied; CLAMP used $P = 1$ on Bail, Credit, Pokec-z, Pokec-n and AMiner-S and $P = 10$ on German, NBA and Facebook, whereas CLAMP-RT uses $P = 10$ everywhere. ^{c}: reads per-client fairness quantities or group rates in the clear. PUFFLE and FedFDP barely learn at their privacy level (AUC near 0.5).

	German		Bail		Credit		Pokec-z	
	AUC	DPD	AUC	DPD	AUC	DPD	AUC	DPD
FedAvg-GAT	0.76±0.05	0.110±0.058	0.80±0.04	0.019±0.011	0.77±0.01	0.078±0.010	0.74±0.02	0.031±0.013
FairFed^{c}	0.75±0.05	0.049±0.039	0.80±0.03	0.012±0.007	0.75±0.01	0.008±0.005	0.64±0.05	0.004±0.003
F^{2}GNN^{c}	0.73±0.02	0.042±0.030	0.78±0.03	0.012±0.008	0.76±0.01	0.006±0.007	0.74±0.02	0.008±0.007
FairGFL	0.74±0.05	0.081±0.070	0.78±0.03	0.015±0.006	0.76±0.01	0.072±0.009	0.66±0.02	0.041±0.011
FedGraph-Fair	0.66±0.09	0.080±0.051	0.76±0.03	0.010±0.007	0.75±0.01	0.075±0.014	0.64±0.03	0.039±0.013
FedFACT^{c}	0.72±0.04	0.078±0.064	0.80±0.03	0.013±0.008	0.75±0.01	0.070±0.016	0.63±0.04	0.024±0.014
PUFFLE	0.49±0.08	0.011±0.007	0.50±0.06	0.003±0.003	0.46±0.13	0.016±0.008	0.55±0.11	0.008±0.009
FedFDP	0.53±0.09	0.014±0.021	0.51±0.03	0.004±0.003	0.54±0.11	0.017±0.009	0.64±0.11	0.008±0.008
FairGNN	0.57±0.06	0.111±0.082	0.56±0.09	0.019±0.020	0.67±0.09	0.100±0.055	0.54±0.08	0.005±0.013
FairSIN	0.72±0.05	0.092±0.062	0.79±0.03	0.025±0.008	0.75±0.00	0.070±0.012	0.66±0.05	0.026±0.010
ClipMean (no controller)	0.73±0.06	0.088±0.069	0.78±0.03	0.016±0.009	0.77±0.01	0.071±0.009	0.75±0.02	0.042±0.011
CLAMP (sums channel)	0.72±0.06	0.058±0.044	0.78±0.03	0.016±0.009	0.76±0.01	0.018±0.015	0.75±0.02	0.026±0.004
CLAMP-GCN (sums channel)	0.73±0.05	0.065±0.044	0.79±0.02	0.015±0.006	0.75±0.01	0.014±0.013	0.72±0.02	0.028±0.006
CLAMP-RT, no bias correction	0.51±0.06	0.602±0.084	0.77±0.04	0.014±0.008	0.76±0.01	0.049±0.041	0.76±0.03	0.032±0.011
CLAMP-RT-GCN	0.71±0.07	0.172±0.198	0.79±0.03	0.014±0.007	0.76±0.01	0.059±0.017	0.74±0.02	0.041±0.009
CLAMP-RT	0.72±0.07	0.172±0.189	0.78±0.03	0.015±0.010	0.77±0.01	0.061±0.017	0.76±0.03	0.037±0.011

	Pokec-n		AMiner-S		NBA		Facebook	
	AUC	DPD	AUC	DPD	AUC	DPD	AUC	DPD
FedAvg-GAT	0.75±0.02	0.034±0.017	0.94±0.00	0.007±0.004	0.81±0.06	0.099±0.059	0.77±0.05	0.078±0.048
FairFed^{c}	0.62±0.06	0.006±0.004	0.94±0.00	0.005±0.003	0.80±0.05	0.081±0.062	0.76±0.03	0.049±0.031
F^{2}GNN^{c}	0.74±0.03	0.008±0.007	0.94±0.01	0.005±0.004	0.78±0.13	0.109±0.072	0.76±0.05	0.069±0.040
FairGFL	0.65±0.03	0.044±0.007	0.94±0.01	0.003±0.002	0.76±0.04	0.078±0.044	0.75±0.05	0.080±0.040
FedGraph-Fair	0.59±0.05	0.024±0.015	0.93±0.01	0.003±0.003	0.77±0.10	0.084±0.043	0.74±0.05	0.094±0.037
FedFACT^{c}	0.59±0.06	0.012±0.011	0.92±0.01	0.008±0.004	0.76±0.04	0.142±0.082	0.74±0.05	0.055±0.042
PUFFLE	0.57±0.11	0.010±0.006	0.49±0.03	0.006±0.005	0.37±0.19	0.014±0.012	0.53±0.10	0.009±0.006
FedFDP	0.66±0.08	0.010±0.008	0.49±0.03	0.006±0.004	0.39±0.20	0.015±0.010	0.54±0.08	0.013±0.009
FairGNN	0.50±0.00	0.000±0.000	0.63±0.08	0.098±0.035	0.77±0.08	0.097±0.044	0.51±0.05	0.024±0.041
FairSIN	0.64±0.06	0.043±0.014	0.92±0.01	0.005±0.003	0.73±0.08	0.087±0.059	0.74±0.05	0.070±0.033
ClipMean (no controller)	0.75±0.02	0.052±0.016	0.94±0.01	0.005±0.004	0.78±0.08	0.093±0.068	0.77±0.06	0.086±0.043
CLAMP (sums channel)	0.76±0.02	0.028±0.012	0.94±0.01	0.004±0.004	0.80±0.08	0.076±0.054	0.76±0.06	0.069±0.034
CLAMP-GCN (sums channel)	0.71±0.02	0.027±0.008	0.94±0.00	0.002±0.002	0.78±0.07	0.077±0.044	0.76±0.04	0.078±0.030
CLAMP-RT, no bias correction	0.77±0.02	0.041±0.014	0.94±0.01	0.007±0.005	0.68±0.10	0.203±0.101	0.65±0.08	0.114±0.063
CLAMP-RT-GCN	0.72±0.03	0.046±0.012	0.94±0.01	0.002±0.002	0.76±0.08	0.116±0.065	0.75±0.06	0.073±0.025
CLAMP-RT	0.76±0.02	0.047±0.015	0.94±0.01	0.004±0.004	0.79±0.08	0.067±0.050	0.76±0.06	0.080±0.054

Table 3: Statistics channel of CLAMP-RT in the honest runs ($K = 5$, $b = 1$, $\varepsilon = 8$, $R = 10$ releases, 10 seeds). $n_{0,\min}$ and σ_d: range over seeds. Radius: $\sqrt{\tau^2 + 2\sigma_d^2} + r_R$ of Theorem 6 at the median σ_d and $\beta = 0.05$ (vacuous when ≥ 1). $\widehat{D} > 0$: share of releases with a non-zero bias-corrected magnitude. λ^T: mean final multiplier. Right block: mean test DPD and AUC of CLAMP (sums channel) and CLAMP-RT, with the two-sided Wilcoxon signed-rank p-value paired by seed, Holm-adjusted across the eight datasets per metric.

Dataset	$n_{0,\min}$	σ_d	Radius	$\widehat{D} > 0$	λ^T	DPD: CLAMP	RT	p	AUC: CLAMP	RT	p
German	1–2	0.672–1.345	8.48	0.31	2.27	0.058	0.172	0.322	0.721	0.719	1.000
Bail	13–26	0.052–0.103	0.59	0.29	0.09	0.016	0.015	0.773	0.778	0.776	1.000
Credit	19–29	0.046–0.071	0.40	0.42	0.08	0.018	0.061	0.016	0.760	0.769	0.016
Pokec-z	42–105	0.013–0.032	0.19	0.45	0.03	0.026	0.037	0.082	0.753	0.757	0.293
Pokec-n	41–64	0.021–0.033	0.19	0.59	0.06	0.028	0.047	0.016	0.758	0.764	1.000
AMiner-S	25–44	0.031–0.054	0.29	0.28	0.02	0.004	0.004	1.000	0.942	0.941	0.260
NBA	1–1	1.345–1.345	8.48	0.29	2.86	0.076	0.067	1.000	0.796	0.788	1.000
Facebook	1–2	0.672–1.345	8.48	0.28	2.85	0.069	0.080	1.000	0.762	0.764	1.000

Where the budget binds, CLAMP-RT controls the gap less tightly than CLAMP. On Credit, Pokec-z and Pokec-n the unconstrained clipped model has a gap above $\tau = 0.03$ (ClipMean 0.071, 0.042 and 0.052). CLAMP brought it to 0.018, 0.026 and 0.028. CLAMP-RT reaches 0.061, 0.037 and 0.047, higher than CLAMP on all three graphs; after Holm correction across the eight datasets the difference is significant on Credit and Pokec-n ($p = 0.016$) but not on Pokec-z ($p = 0.082$). AUC is equal or slightly higher (0.769 against 0.760 on Credit, Holm-adjusted $p = 0.016$; 0.757 against 0.753 on Pokec-z; 0.764 against 0.758 on Pokec-n, neither significant). Against ClipMean the reduction is small (0.071 to 0.061 on Credit, 0.052 to 0.047 on Pokec-n), and on the two Pokec graphs plain FedAvg-GAT has a lower mean gap (0.031 and 0.034). In the honest setting, CLAMP-RT is therefore a weaker fairness controller than CLAMP. The logs and the noise-free runs suggest several contributing causes, which we did not isolate. First, the multiplier stays small: its final value averages 0.03–0.08 on these graphs (Table 3) against 0.17–0.89 for CLAMP on Credit; CLAMP-RT updates it ten times instead of a hundred. Second, the noise on one trimmed scalar is large relative to the gap: σ_d is 0.046–0.071 on Credit, larger than the budget, and after bias correction only 42–59% of the releases have a non-zero magnitude on these three graphs. Third, the controller targets a trimmed mean of per-client gaps, not the pooled gap. The noise-free runs of Table 12 (three seeds) help to separate these. On Pokec-z and Pokec-n, noise-free per-round releases reach 0.017 and 0.019, below CLAMP, and noise-free $P = 10$ gives 0.031 and 0.029: there, noise and the release period account for the shortfall, and the proxy

appears adequate. On Credit, noise-free $P = 10$ gives 0.048, no better than with noise at $\varepsilon = 8$ (0.044 on the same seeds), and noise-free $P = 1$ gives 0.035, still above CLAMP (0.007 on the same three seeds at $K = 5$, Table 5). On Credit, whose protected group is 9% of the nodes, the per-client proxy is therefore the likely main cause, but we have not isolated it. The baselines that read per-client fairness quantities in the clear reach lower gaps than both protocols (F^2GNN 0.006–0.008 on these graphs), as expected.

The bias correction costs control where the budget binds. The bias correction was introduced for the small graphs. Where the budget binds it makes the gap worse: without it, CLAMP-RT reaches 0.049, 0.032 and 0.041 on Credit, Pokec-z and Pokec-n, against 0.061, 0.037 and 0.047 with it (Table 2), because the corrected magnitude is zero in roughly half of the releases. We keep a single variant for all datasets, and report both.

Where the budget does not bind. On Bail and AMiner-S the unconstrained gap is already below τ (0.016 and 0.005), and CLAMP-RT, CLAMP and ClipMean are within seed noise of each other (CLAMP-RT 0.015 and 0.004).

Small graphs: the gap channel carries almost no signal. On German, NBA and Facebook some client has at most 20–40 validation nodes, so $n_{0,\min} = 1$ or 2 and $\sigma_d = 0.67$–1.34, far above the honest trimmed gaps, whose mean magnitude before noise is 0.04–0.17. At $\varepsilon = 8$ the released gap is essentially noise. Without bias correction the inflated magnitude $|\tilde{g}|$ drove the multiplier up, no release round after the warm-up was feasible in 10, 9 and 10 of 10 runs (German, NBA, Facebook), and the returned last model was damaged (German AUC 0.51, gap 0.602). With the correction, $\widehat{D} = 0$ in about 70% of the releases, but the remaining releases still push the multiplier to 2.3–2.9 on average, with a random direction. The correction also loosens the feasibility test to $|\tilde{g}| \le \sqrt{\tau^2 + 2\sigma_d^2} \approx 0.95$–$1.9$, which almost every release passes, so the output rule returned a model from the last release rounds (round 91 in 7 of 10 runs) instead of falling back. Much of the improvement over the uncorrected variant on these graphs therefore comes from the output rule, not from better control. The result is a gap of 0.067 on NBA and 0.080 on Facebook, close to CLAMP (0.076, 0.069) and ClipMean (0.093, 0.086), and 0.172 ± 0.189 on German, worse and far more variable than CLAMP (0.058 ± 0.044; Holm-adjusted $p = 0.32$). We do not consider CLAMP-RT usable on graphs of this size at this privacy level. (Without the output-rule warm-up,

Table 4: Falsified gap reports against CLAMP-RT: test DPD, mean±sd over 3 seeds, for f lying clients out of K. Trimming level $b = 1$ ($K = 5$) and $b = 2$ ($K = 10$); rows with $f > b$ (marked $*$) are beyond the breakdown point. "Honest" is the same configuration with no liars. Context columns, for the honest run: ClipMean is clipped aggregation without a controller on the same seeds and K (– where no such run exists); λ^T is the mean final multiplier of CLAMP-RT and $\widehat{D} > 0$ the share of its releases with a non-zero corrected magnitude. AUC is in the supplementary material.

Dataset	K	f	ClipMean	λ^T	$\widehat{D} > 0$	Honest	hide	inflate	inlier	zero
Bail	5	1	0.023	0.081	0.30	0.023±0.004	0.023±0.004	0.023±0.004	0.023±0.004	0.023±0.004
		2*					0.027±0.014	0.027±0.013	0.023±0.004	0.023±0.004
	10	1	0.009	0.027	0.23	0.008±0.002	0.009±0.002	0.009±0.002	0.008±0.002	0.009±0.002
		2					0.009±0.002	0.008±0.002	0.008±0.002	0.008±0.002
		3*					0.006±0.007	0.009±0.004	0.009±0.002	0.008±0.002
Credit	5	1	0.071	0.137	0.53	0.043±0.019	0.052±0.015	0.044±0.019	0.052±0.014	0.051±0.015
		2*					0.276±0.029	0.315±0.034	0.059±0.017	0.062±0.011
	10	1	–	0.116	0.57	0.038±0.008	0.041±0.007	0.036±0.008	0.040±0.007	0.041±0.007
		2					0.043±0.005	0.032±0.007	0.043±0.005	0.043±0.005
		3*					0.112±0.030	0.234±0.054	0.045±0.006	0.048±0.007
Pokec-z	5	1	0.044	0.013	0.30	0.041±0.004	0.040±0.006	0.041±0.004	0.041±0.004	0.042±0.004
		2*					0.281±0.024	0.088±0.040	0.043±0.004	0.044±0.005
	10	1	0.045	0.012	0.53	0.046±0.008	0.046±0.008	0.045±0.008	0.045±0.008	0.046±0.008
		2					0.046±0.008	0.043±0.008	0.046±0.008	0.046±0.008
		3*					0.116±0.012	0.015±0.016	0.046±0.008	0.046±0.008
Pokec-n	5	1	0.044	0.009	0.37	0.042±0.012	0.045±0.012	0.040±0.009	0.047±0.011	0.046±0.011
		2*					0.304±0.030	0.180±0.011	0.044±0.010	0.045±0.012
	10	1	–	0.010	0.53	0.046±0.001	0.046±0.002	0.044±0.001	0.046±0.000	0.046±0.001
		2					0.047±0.002	0.043±0.001	0.047±0.002	0.047±0.002
		3*					0.127±0.006	0.030±0.031	0.047±0.003	0.048±0.003

the CLAMP runs give 0.052 on German and 0.067 on Facebook; the other values are unchanged.)

Backbone. CLAMP-RT-GCN behaves like CLAMP-RT on every dataset (Table 2).

7.2. Falsified statistics

Up to the breakdown point, the attacks do not move the outcome away from the honest level. Table 4 and Figure 1 report the four gap attacks. For every dataset, K, attack and $f \leq b$, the mean test gap stays within 0.009 of the honest run with the same seeds, which is less than its seed-to-seed standard deviation; the largest change is on Credit with $K = 5$ and one hiding client (0.043 to 0.052). AUC changes by less than 0.01 (Table S1). In all 1,440 attacked release rounds with $f \leq b$ the pre-noise trimmed mean lay inside the interval of Theorem 5, as it must; this checks the implementation, not the theorem.

Table 5: Sums channel (CLAMP) versus trimmed channel (CLAMP-RT) under falsified statistics: test DPD, mean over 3 seeds. CLAMP: swap falsification of the four group sums with the public cap ($P = 1$). CLAMP-RT: largest mean DPD over the four gap attacks ($P = 10$). $f = 2$ at $K = 5$ exceeds the breakdown point $b = 1$ of CLAMP-RT.

		CLAMP (sums)			CLAMP-RT (trimmed)		
Dataset	K	$f = 0$	$f = 1$	$f = 2$	$f = 0$	$f = 1$	$f = 2$
Bail	5	0.023	0.023	0.023	0.023	0.023	0.027
	10	0.010	0.009	0.009	0.008	0.009	0.009
Credit	5	0.007	0.070	0.209	0.043	0.052	0.315
	10	0.011	0.065	0.065	0.038	0.041	0.043
Pokec-z	5	0.026	0.046	0.257	0.041	0.042	0.281
	10	0.027	0.045	0.045	0.046	0.046	0.046
Pokec-n	5	0.027	0.042	0.187	0.042	0.047	0.304
	10	0.032	0.049	0.050	0.046	0.046	0.047

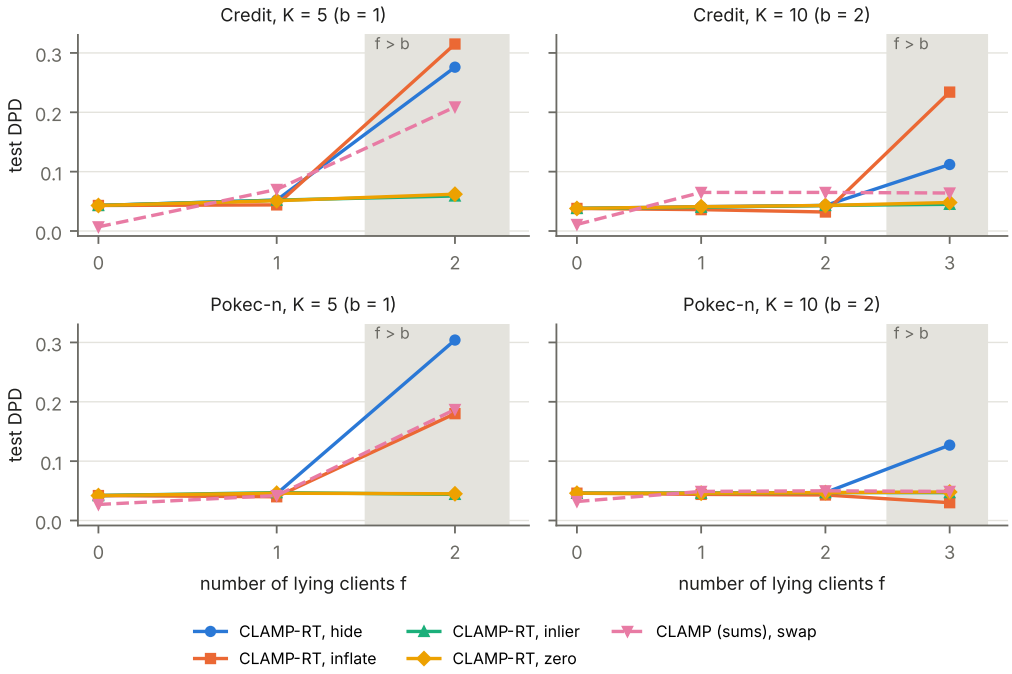


Figure 1: Test DPD against the number of lying clients f (mean over 3 seeds). Shaded: $f > b$, beyond the breakdown point of the trimmed channel. Dashed: CLAMP (sums channel, $P = 1$) under the swap attack.

This result must be read together with the context columns of Table 4. What the attacks fail to do is move the outcome away from the honest run; whether that honest run is itself well controlled is a separate question. On Credit it is: at $K = 5$ the honest gap is 0.043 against 0.071 for ClipMean on the same seeds, the final multiplier is 0.14 and 53% of the releases have $\widehat{D} > 0$. On the Pokec graphs it is not: the honest gap is 0.041–0.046, the same as ClipMean where it was run (0.044 at $K = 5$, 0.045 for Pokec-z at $K = 10$), and the final multiplier is about 0.01. There the controller barely acts in the honest run, so an attack that leaves the gap unchanged shows only that the liars could not *create* a problem, not that CLAMP-RT would keep a large gap under control against them. On Bail the budget does not bind and the controller is inactive. Section 7.3 adds an adaptive attacker, repeats the study with per-round releases, where the controller is more active, and combines gap and update attacks.

Beyond the breakdown point, the hiding and inflating attacks succeed. With $f = b + 1$ one false value survives the trimming. The *hide* attack, which reports a gap of full size with the opposite sign, then flips the sign of the estimate, and the controller pushes the model the wrong way: the gap rises to 0.276–0.304 with $K = 5$ and to 0.112–0.127 with $K = 10$ on Credit and the Pokec graphs, above the unconstrained level. The *inflate* attack forces over-correction; on Credit it raises the gap to 0.315 ($K = 5$) and 0.234 ($K = 10$) and costs 0.09–0.14 AUC, on the Pokec graphs it raises the gap with $K = 5$ (0.088 on Pokec-z, 0.180 on Pokec-n), whereas with $K = 10$ it lowers the gap below the honest level (0.015 and 0.030) at a cost of about 0.01 AUC. In all runs beyond the breakdown point where the hide or inflate attack succeeded, the multiplier grew well beyond its honest level (final values 0.18–2.04, against at most 0.14 in the honest runs), and in 35 of the 96 runs beyond the breakdown point no release after the warm-up was feasible, so the last model was returned. The *inlier* and *zero* attacks, whose claims stay close to the honest values, have no clear effect even beyond the breakdown point (at most 0.062 against 0.043 on Credit). On Bail, where the budget does not bind, no attack has an effect.

Comparison with the sums channel. Table 5 compares the two channels. With the sums channel a single liar out of ten raised the gap from 0.011 to 0.065 on Credit, from 0.027 to 0.045 on Pokec-z and from 0.032 to 0.049 on Pokec-n, i.e. to roughly the level of training without control, and additional liars did not

Table 6: Adaptive gap attack (*opt*: all liars report the common value that minimises the signed trimmed mean, Section 6) at $P = 10$: test DPD, mean±sd over 3 seeds. Honest: same seeds without liars (Table 4). $*$: $f > b$. AUC in Table S6.

Dataset	K	Honest	$f = 1$	$f = 2$	$f = 3$
Bail	5	0.023±0.004	0.023±0.004	0.028±0.014*	–
	10	0.008±0.002	0.009±0.002	0.009±0.002	0.006±0.007*
Credit	5	0.043±0.019	0.052±0.014	0.275±0.028*	–
	10	0.038±0.008	0.041±0.007	0.043±0.005	0.112±0.030*
Pokec-z	5	0.041±0.004	0.042±0.006	0.279±0.029*	–
	10	0.046±0.008	0.046±0.008	0.046±0.008	0.116±0.013*
Pokec-n	5	0.042±0.012	0.045±0.011	0.304±0.031*	–
	10	0.046±0.001	0.046±0.002	0.047±0.003	0.126±0.006*

Table 7: Gap attacks with per-round releases ($P = 1$, $R = 100$, $K = 10$, $b = 2$, $\varepsilon = 8$): test DPD, mean over 3 seeds. Left block: honest runs at $P = 1$ and, for comparison, at $P = 10$ (Table 4) and ClipMean on the same seeds where it was run; λ^T and $\widehat{D} > 0$ as in Table 4. $f \le b$: largest mean DPD over the four attacks. $f = 3 > b$: each attack. Full table with sd and AUC: Table S7.

	Honest					worst, $f \le b$		$f = 3$ (beyond breakdown)			
Dataset	$P = 1$	λ^T	$\widehat{D} > 0$	$P = 10$	ClipMean	$f = 1$	$f = 2$	hide	inflate	inlier	opt
Credit	0.053	1.68	0.34	0.038	–	0.059	0.059	0.096	0.066	0.053	0.095
Pokec-z	0.035	0.30	0.37	0.046	0.045	0.039	0.041	0.161	0.035	0.042	0.161
Pokec-n	0.037	0.30	0.36	0.046	–	0.041	0.043	0.164	0.010	0.046	0.164

change this. With the trimmed channel, one or two liars out of ten leave the gap at the honest level. The comparison has to be read with Table 2 in mind: because the honest level of CLAMP-RT is itself higher, the attacked gap of CLAMP-RT is lower than the attacked gap of CLAMP only on Credit (0.041 against 0.065 for $f = 1$, $K = 10$); on the Pokec graphs the two are equal (0.046 against 0.045 and 0.049). What CLAMP-RT gains is predictability: up to b liars and against these fixed attacks, its fairness under attack was its fairness without attack, whatever that level was. With $K = 5$ and two liars both channels fail.

7.3. Adaptive, per-round and combined attacks

Adaptive attack. The *opt* attack lets all liars report the common value that minimises the signed noise-free trimmed mean, given all honest gaps and the public b; it is the myopically optimal single hiding claim. Table 6 reports it at $P = 10$. For $f \le b$ it leaves the gap at the honest level, as the fixed

attacks did: on Credit with $K = 10$ it gives 0.041 and 0.043 for $f = 1, 2$ against 0.038 honest, on Pokec-z 0.046 against 0.046, and the largest change over all settings is again Credit with $K = 5$ and $f = 1$ (0.052 against 0.043). Beyond the breakdown point it does as much damage as the hiding attack: 0.275, 0.279 and 0.304 on Credit, Pokec-z and Pokec-n with $K = 5$, $f = 2$, and 0.112–0.126 with $K = 10$, $f = 3$. For $f \leq b$ the optimal claim is an honest-range value, so by Theorem 5 it can only shift the estimate within the interval; for $f > b$ it coincides in effect with the extreme claim of the hiding attack. The attack is a concrete strong attacker, not a proof of the worst case over all adaptive strategies: it uses one common value per release, it is myopic across releases, and it does not combine falsified gaps with poisoned updates.

Per-round releases. With $P = 10$ the multiplier on the Pokec graphs stayed near zero, so the attack study there could not show whether a working controller survives liars. Table 7 repeats it with $P = 1$ ($R = 100$ releases, hence a larger σ_d of 0.049 on the Pokec graphs and 0.112 on Credit at $K = 10$). The controller is now active: the final multiplier of the honest runs is 0.30 on Pokec-z and 0.30 on Pokec-n, against 0.01 and 0.01 at $P = 10$, and 1.68 on Credit. On the Pokec graphs this lowers the honest gap from 0.046 to 0.035 and 0.037; on Pokec-z this is below ClipMean on the same seeds (0.045), so control has some effect, but the gap stays above the budget. On Credit the honest gap rises from 0.038 to 0.053: the larger noise of 100 releases drives the multiplier without bringing the gap down. Per-round releases therefore improve the honest level on the Pokec graphs and worsen it on Credit, consistent with the $K = 5$ sweep of Table 12.

Under attack with $f \leq b$, the outcome again stays at the honest level: the largest mean gap over the hide, inflate, inlier and opt attacks is within 0.006 of the honest run on all three graphs. With an active controller this is a more meaningful result than at $P = 10$: on the Pokec graphs the liars did not undo the reduction from 0.046 to 0.035–0.037 that the controller achieved. It remains a small reduction, and on Credit the controlled level is worse than at $P = 10$. Beyond the breakdown point ($f = 3$) the hiding and adaptive attacks raise the gap to 0.095–0.096 on Credit and 0.161–0.164 on the Pokec graphs, above ClipMean. The inflating attack at $f = 3$ does not raise the gap (0.066 on Credit, 0.035 on Pokec-z, 0.010 on Pokec-n), but on Credit it drives the multiplier to 6.85 on average and lowers AUC from 0.753 to 0.724 $\pm$ 0.055. No ClipMean reference exists for Credit and Pokec-n at $K = 10$.

Table 8: Combined attack ($K = 10$, $b = 2$, $P = 10$): the same f clients poison their updates and report the adaptive *opt* gap. Test DPD, mean over 3 seeds, as update-only → combined. Update-only references: Table S2 (Bail, Pokec-z) and dedicated runs (Credit). $f = 3 > b$. Full table with sd and AUC: Table S8.

Dataset	Update attack	$f = 1$	$f = 2$	$f = 3$
Bail	Fair. poison	0.010 → 0.010	0.009 → 0.010	0.015 → 0.021
	Sign flip	0.009 → 0.009	0.009 → 0.009	0.009 → 0.002
	ALIE	0.010 → 0.010	0.007 → 0.007	0.006 → 0.005
Credit	Fair. poison	0.040 → 0.044	0.045 → 0.051	0.051 → 0.138
	Sign flip	0.037 → 0.041	0.036 → 0.046	0.038 → 0.112
	ALIE	0.037 → 0.040	0.037 → 0.042	0.041 → 0.112
Pokec-z	Fair. poison	0.059 → 0.062	0.118 → 0.122	0.151 → 0.212
	Sign flip	0.043 → 0.042	0.032 → 0.032	0.025 → 0.062
	ALIE	0.045 → 0.046	0.042 → 0.042	0.036 → 0.106

Combined update and gap attacks. Table 8 lets the same f clients poison their updates (fairness poisoning, sign flipping or ALIE) and report the adaptive *opt* gap ($K = 10$, $P = 10$). For $f \leq b = 2$ the combined attack is close to the update attack alone: the largest increase is 0.010 (Credit, sign flipping, $f = 2$: 0.036 to 0.046), on Pokec-z fairness poisoning gives 0.118 alone and 0.122 combined, and on Bail every change is within seed noise. This does not mean the protocol resists the update attack: fairness poisoning on Pokec-z already raises the gap to 0.118 without any false gap, as in Section 7.4; the gap channel adds little on top. For $f = 3 > b$ the combination breaks the gap channel as the gap attack alone did: on Credit the gap rises from 0.041, 0.038 and 0.051 (ALIE, sign flipping, fairness poisoning alone) to 0.112, 0.112 and 0.138, and on Pokec-z from 0.036, 0.025 and 0.151 to 0.106, 0.062 and 0.212. On Bail, where the budget does not bind, $f = 3$ changes little (fairness poisoning 0.015 to 0.021). The hiding and adaptive gap claims give the same results under fairness poisoning (Table S8). The breakdown at $f > b$ is what Theorem 5, whose condition is $f \leq b$, leaves open; the experiment shows that it is reached in practice, not that the theorem is tight.

7.4. Update attacks and fairness poisoning

The trimmed channel does not change the update channel, and the results confirm it. Under the four update attacks that target utility, CLAMP-RT behaves like the clipped mean (Table S2): scaling has no effect because scaled

Table 9: Fairness poisoning through updates ($K = 10$): test DPD (AUC in parentheses), mean over 3 seeds. CLAMP (sums channel) uses $P = 1$; † needs individual updates and cannot run under secure aggregation. CLAMP-RT uses $P = 10$.

Dataset	f	FedAvg	Median†	TrMean†	Krum†	ClipMean	CLAMP	CLAMP-RT
Pokec-z	0	0.035 (0.75)	0.033 (0.75)	0.035 (0.74)	0.033 (0.74)	0.045 (0.76)	0.028 (0.76)	0.046 (0.77)
	1	0.062 (0.75)	0.046 (0.74)	0.047 (0.73)	0.035 (0.74)	0.065 (0.76)	0.025 (0.76)	0.059 (0.77)
	2	0.124 (0.71)	0.108 (0.72)	0.146 (0.70)	0.034 (0.72)	0.137 (0.75)	0.037 (0.72)	0.118 (0.75)
	3	0.137 (0.70)	0.148 (0.72)	0.186 (0.69)	0.035 (0.73)	0.172 (0.74)	0.049 (0.71)	0.151 (0.74)
	4	0.137 (0.69)	0.111 (0.72)	0.201 (0.71)	0.086 (0.71)	0.125 (0.73)	0.047 (0.71)	0.106 (0.73)
Bail	0	0.011 (0.76)	0.007 (0.73)	0.013 (0.75)	0.003 (0.72)	0.009 (0.75)	0.011 (0.75)	0.009 (0.74)
	1	0.008 (0.75)	0.008 (0.72)	0.014 (0.74)	0.005 (0.72)	0.011 (0.74)	0.012 (0.74)	0.010 (0.74)
	2	0.014 (0.72)	0.009 (0.72)	0.010 (0.73)	0.009 (0.71)	0.010 (0.72)	0.011 (0.72)	0.009 (0.72)
	3	0.018 (0.71)	0.010 (0.72)	0.012 (0.73)	0.005 (0.71)	0.016 (0.72)	0.009 (0.72)	0.015 (0.71)
	4	0.021 (0.70)	0.013 (0.71)	0.014 (0.72)	0.009 (0.71)	0.023 (0.71)	0.014 (0.72)	0.023 (0.71)

updates are clipped; sign flipping lowers AUC gradually from 0.743 to 0.661 on Bail and from 0.768 to 0.710 on Pokec-z as f grows from 0 to 4; ALIE and IPM cost at most 0.033 AUC.

Fairness poisoning is different (Table 9). On Pokec-z it raises the gap of CLAMP-RT to 0.118, 0.151 and 0.106 for $f = 2, 3, 4$, close to ClipMean (0.137, 0.172, 0.125) and FedAvg (0.124–0.137). CLAMP with per-round releases held the same attack at 0.037–0.049. In this respect CLAMP-RT is a regression. The poisoning clients report their true gaps, so the gap channel does see the growing gap, but the multiplier of CLAMP-RT reached at most 0.39 at the end of these runs, whereas that of CLAMP reached 0.56–4.04 in eight of its nine runs with $f \geq 2$. We attribute the difference mainly to the release period ($P = 10$ gives the controller ten updates instead of a hundred), but we did not run CLAMP-RT with $P = 1$ under this attack, so this explanation is untested. Among the rules that need individual updates, only Krum held the gap (0.034–0.035 up to $f = 3$). On Bail the attack raises the gap of every method by at most 0.014.

7.5. Cost of the secure computation

We implemented the server step (validity checks, masked update sum, sort, trimming and noise) in CrypTen [19]: semi-honest two-party computation over $\mathbb{Z}_{2^{64}}$ with 16 fractional bits, Beaver triples from a trusted dealer, two processes on one machine, and an odd-even transposition sorting network (K passes). Table 10 reports one release round.

The gap channel itself does not depend on the model size. It takes 64, 109 and 199 communication rounds and 9.3, 30.2 and 106.7 KB for $K = 5$, 10

Table 10: Measured cost of one release round of the server-side protocol in semi-honest two-party computation (CrypTen, two processes on one machine, local network; median of 3 repetitions). Bytes are those sent by one server as reported by CrypTen; rounds are communication rounds. Update channel: norm checks and masked sum of K updates of dimension d. Gap channel: range checks, sort, trimmed mean and noise addition. Err: largest absolute fixed-point error of the trimmed mean (noise switched off), and of the update sum in the update block. The gap channel uses the validity bits of the update channel (a rejected update also removes the client's gap), so the full release round is the sum of the two blocks. Client-to-server input sharing and Beaver-triple generation are not included.

			Update channel				Gap channel				Full round	
K	b	d	MB	rounds	s	Err	KB	rounds	s	Err	MB	s
5	1	10^4	2.56	12	0.79	4.5×10^{-7}	9.3	64	0.33	5.9×10^{-6}	2.57	1.12
5	1	10^5	25.60	12	1.44	3.2×10^{-3}	9.3	64	0.30	1.0×10^{-5}	25.61	1.74
5	1	10^6	256.00	12	3.12	1.0×10^{-3}	9.3	64	0.33	1.5×10^{-5}	256.01	3.44
10	2	10^4	4.96	12	0.88	9.1×10^{-7}	30.2	109	0.54	1.0×10^{-5}	4.99	1.42
10	2	10^5	49.60	12	1.33	3.2×10^{-3}	30.2	109	0.51	1.1×10^{-5}	49.63	1.84
10	2	10^6	496.00	12	4.51	1.0×10^{-3}	30.2	109	0.42	6.4×10^{-6}	496.03	4.94
20	4	10^4	9.77	12	0.73	1.6×10^{-6}	106.7	199	0.92	9.7×10^{-6}	9.88	1.65
20	4	10^5	97.61	12	1.47	5.4×10^{-7}	106.7	199	1.12	1.2×10^{-5}	97.72	2.59
20	4	10^6	976.01	12	9.55	1.0×10^{-3}	106.7	199	0.95	1.5×10^{-5}	976.12	10.50

and 20 clients, and 0.30–1.12 s on the local machine. It is not a stand-alone cost, however: a client's gap is used only if its update passes the norm check, so every release round also runs the update channel, which takes 12 rounds for any size and whose traffic grows linearly with $K \cdot d$. A full release round therefore costs 4.99 MB and 1.4 s for $K = 10$, $d = 10^4$, and 976 MB and 10.5 s for $K = 20$, $d = 10^6$, almost all of it in the update channel. The GAT backbones of our experiments have about 5×10^3 (Credit, Bail) to 4×10^4 (Facebook) parameters, computed from the layer shapes ($64 \cdot \text{features} + 4{,}545$ for two GAT layers with four heads of 16 units and a linear output), so $d = 10^4$ is the representative row. Over a wide-area network the round count of the gap channel matters; at 50 ms round-trip time, 199 rounds would add about 10 s per release, a figure we computed and did not measure. The largest fixed-point error of the trimmed mean was 1.5×10^{-5}, about 840 times smaller than the smallest σ_d in our runs (0.0128); the largest error of the update sum was 3.2×10^{-3} (for $d \geq 10^5$), small against the norm bound $C = 1$ but not negligible in every coordinate.

The measurement has clear limits. It is semi-honest, so a server that deviates from the protocol is not covered; ELSA-style malicious security [16]

Table 11: Balanced accuracy of the optimal test that distinguishes two attribute-neighbouring datasets from the released gap (closed form, worst case $|\Delta| = \Delta_{\mathrm{TM}}$; $R = 10$ releases, $\delta = 10^{-5}$). $z = \sigma_d/\Delta_{\mathrm{TM}}$ from the RDP calibration. Both servers honest: one release with variance $2\sigma_d^2$. One server colluding: it removes its own noise. All releases: the colluding server combines all R releases of a target whose contribution is unchanged. Bound: $(e^\varepsilon + \delta)/(1 + e^\varepsilon)$.

ε	z	Both honest, one release	One server, one release	One server, all R	Bound
1	12.79	0.511	0.516	0.549	0.7311
2	6.80	0.521	0.529	0.592	0.8808
4	3.66	0.538	0.554	0.667	0.9820
8	2.02	0.570	0.598	0.783	0.9997

Table 12: Privacy budget ε of the gap channel and release period P ($K = 5$, 3 seeds): test DPD, mean±sd. Row ∞: no noise on the gap ($\sigma_d = 0$, where the bias correction is the identity); these runs were made with $P \in \{1, 10\}$ only. All other experiments use $\varepsilon = 8$ and $P = 10$.

	Credit			Pokec-z			Pokec-n		
ε	$P = 1$	$P = 5$	$P = 10$	$P = 1$	$P = 5$	$P = 10$	$P = 1$	$P = 5$	$P = 10$
1	0.057±0.022	0.057±0.007	0.063±0.005	0.051±0.027	0.056±0.012	0.046±0.007	0.055±0.039	0.055±0.026	0.048±0.009
2	0.056±0.023	0.060±0.004	0.066±0.006	0.046±0.023	0.048±0.005	0.045±0.006	0.049±0.024	0.051±0.017	0.044±0.007
4	0.061±0.019	0.063±0.003	0.049±0.027	0.030±0.004	0.045±0.002	0.039±0.006	0.032±0.005	0.043±0.004	0.043±0.012
8	0.090±0.056	0.062±0.003	0.044±0.019	0.028±0.002	0.038±0.006	0.041±0.003	0.029±0.003	0.041±0.011	0.043±0.014
∞	0.035±0.010	–	0.048±0.008	0.017±0.007	–	0.031±0.004	0.019±0.007	–	0.029±0.006

would cost more. The trusted dealer and the clients' input sharing are not included. The noise was switched off in the error measurement, and the noise of the protocol is a continuous Gaussian sampled in floating point and then shared, not a discrete mechanism over the ring. A sorting network with lower depth than odd-even transposition would reduce the round count for larger K; we did not implement one. Finally, the training experiments did not run inside CrypTen; they execute the same arithmetic in plaintext.

7.6. Privacy

What the released gap reveals. For a single release, the best test between two neighbouring datasets observes $\tilde{g}$ with mean TM_b or $\mathrm{TM}_b + \Delta$, $|\Delta| \le \Delta_{\mathrm{TM}}$, and noise $\mathcal{N}(0, 2\sigma_d^2)$; its balanced accuracy is $\Phi(\Delta/(2\sqrt{2}\sigma_d)) \le \Phi(1/(2\sqrt{2}z))$, independent of the dataset because $\sigma_d = z\Delta_{\mathrm{TM}}$. Table 11 evaluates it. At $\varepsilon = 8$ it is 0.570 with both servers honest, 0.598 if one server colludes and removes its own noise, and 0.783 if the colluding server also combines all ten releases of a node whose contribution does not change, the worst case the accounting covers. The (ε, δ) bound, $(e^\varepsilon + \delta)/(1 + e^\varepsilon) = 0.9997$, is

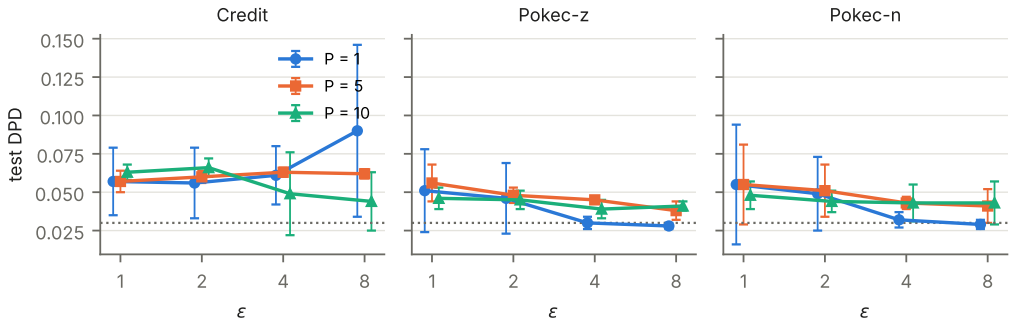


Figure 2: Test DPD of CLAMP-RT against the privacy budget of the gap channel for three release periods (mean ± sd over 3 seeds; dotted: budget $\tau = 0.03$).

uninformative at this level; the closed form is the more useful description. These values are for the worst case $|\Delta| = \Delta_{\mathrm{TM}}$; a node in a group much larger than $n_{0,k}$ changes the trimmed mean by less.

Privacy budget and release period. Table 12 and Figure 2 vary ε and P. On Pokec-z and Pokec-n, per-round releases work best once $\varepsilon \geq 4$ (0.028–0.032), matching CLAMP, while $P = 10$ stays at 0.039–0.043; at $\varepsilon \leq 2$ all periods give 0.044–0.056. Without noise ($\varepsilon = \infty$, last row) the Pokec graphs reach 0.017–0.019 at $P = 1$ and about 0.03 at $P = 10$. On Credit no noisy setting reaches the budget: the gap is 0.044–0.090, with the largest values and spread at $P = 1$, $\varepsilon = 8$ (0.090 ± 0.056), and at $\varepsilon \leq 4$ the results are noisy at every period. Even without noise the Credit gap is 0.035 ($P = 1$) and 0.048 ($P = 10$), so on Credit the noise is not the only cause of the shortfall (Section 7.1). AUC is 0.747–0.775 throughout the noisy runs (Table S4). Our uniform choice $P = 10$ is therefore not the best choice on the Pokec graphs. We nevertheless report all other experiments with $P = 10$, because no single period was better on all three graphs; all attack studies and honest runs use $P = 10$, and $K \in \{5, 10\}$ only.

What the model reveals. Theorem 3 does not cover the trained model. A separate probe study examined the final models of FedAvg, ClipMean and CLAMP (last iterate) with a logistic-regression attribute probe on hidden representations: on German the probe reached AUC 0.86–0.87 for all three, against 0.85 for the raw features and 0.91 for an untrained GAT, and on Bail 0.54 against 0.61 for the raw features. We did not repeat the probe for

Table 13: Ablations of the trimmed channel ($K = 5$, $\varepsilon = 8$, $P = 10$, 3 seeds, honest). These runs use the variant without bias correction. σ_d is the per-server noise standard deviation (mean±sd over seeds).

	Credit			Pokec-z		
Variant	AUC	DPD	σ_d	AUC	DPD	σ_d
Default ($b = 1$, $c = 0.05$)	0.755±0.012	0.026±0.017	0.063±0.014	0.754±0.032	0.041±0.001	0.028±0.002
Last iterate (no output rule)	0.750±0.014	0.031±0.023	0.063±0.014	0.748±0.027	0.036±0.001	0.028±0.002
$b = 0$ (plain mean, no robustness)	0.762±0.007	0.027±0.008	0.038±0.008	0.752±0.027	0.039±0.002	0.017±0.001
$b = 2$ (median of $K = 5$)	0.683±0.041	0.168±0.048	0.188±0.042	0.752±0.038	0.039±0.021	0.084±0.005
$c = 0.2$	0.657±0.004	0.236±0.008	0.016±0.004	0.753±0.026	0.033±0.004	0.007±0.000

CLAMP-RT. Its model is trained by the same loop with a different multiplier, and we expect, but have not verified, similar values. Demographic-parity control does not remove the attribute from the representation, and neither protocol claims that it does.

7.7. Ablations

Table 13 varies the trimming level, the floor fraction and the output rule on Credit and Pokec-z (variant without bias correction, three seeds). Trimming costs little in honest runs: the plain mean ($b = 0$) has a smaller σ_d (0.038 against 0.063 on Credit) and a gap within seed noise of the default (0.027 against 0.026). Two settings are clearly bad. With $b = 2$ at $K = 5$ the statistic is the median, its sensitivity triples ($\sigma_d = 0.188$ on Credit), and the Credit gap rises to 0.168 with AUC 0.683. With $c = 0.2$ the noise is smaller ($\sigma_d = 0.016$), but the Credit gap is 0.236 with AUC 0.657. The likely reason is the floor: the protected group is 9% of Credit's nodes, so with $c = 0.2$ it is smaller than $n_{0,k}$ on most clients, its rate is shrunk toward zero, and the estimated gap is biased in one direction; on Pokec-z, whose groups are larger than the floor, $c = 0.2$ works (0.033). The floor fraction must therefore stay below the smallest group share, which is a constraint the federation has to set from public knowledge. Returning the last iterate instead of the stored model changes the Credit gap by less than the seed noise (0.031 ± 0.023 against 0.026 ± 0.017), but on Pokec-z the last iterate is better than the stored model (0.036 ± 0.001 against 0.041 ± 0.001), a difference larger than the seed spread: with the looser feasibility test of the corrected variant and few releases, the output rule does not always pick a better model than the last one.

Discarded variants. Table S5 in the supplementary material lists all development pilots (two seeds each, among seeds 0–2): the initial floor fraction

$c = 0.1$, exponential smoothing of the released gap (EMA, factor 0.3), a multiplier step scaled by the release period, a tighter controller target $\tau_c = 0.02$, and the first runs of the bias correction. None of them was adopted. Some of them were not worse in these small pilots; for example the noisy Credit pilot with $c = 0.1$ and $P = 10$ gave 0.023 against 0.025 for $c = 0.05$, with two seeds each. The adopted configuration is therefore a choice among similar pilots, not a tuned optimum, and the pilots share seeds with the reported runs.

8. Discussion and limitations

What the trimmed channel buys and what it costs. The trimmed channel answers the main weakness of the sums channel: up to b liars cannot move the noise-free fairness estimate outside the range of honest values (Theorem 5), and in our fixed attacks they did not move the test gap away from the honest level. With ten releases that level was close to training without control on the Pokec graphs. With per-round releases the multiplier was active and lowered the Pokec gaps from 0.046 to 0.035–0.037, and liars with $f \leq b$, including the adaptive attacker, did not undo this. The reduction is small and the gap stays above the budget, so the evidence is for robustness of a weak controller, not of a strong one. The price is paid in the honest case. A trimmed mean of K per-client gaps has sensitivity $2/(n_{0,\min}(K - 2b))$, which is set by the smallest client's validation set, whereas the pooled gap of CLAMP had a sensitivity set by pooled counts. At the same ε the trimmed channel is noisier, and with ten releases it controlled the gap less tightly than CLAMP on Credit and the Pokec graphs. A federation that expects no liars has no reason to prefer CLAMP-RT; one that cannot rule out a minority of liars obtains bounded behaviour at a higher, but predictable, gap.

The proxy. The controller acts on a trimmed mean of per-client gaps, not on the pooled gap. Under strongly non-IID partitions the two can differ, and a client whose local gap has the opposite sign to the pooled one pulls the estimate the wrong way even when honest. The noise-free runs indicate that this matters on Credit, where even noise-free per-round releases leave the gap at 0.035, against 0.007 for the pooled channel on the same seeds, whereas on the Pokec graphs the noise-free trimmed channel matches or beats the pooled one. We have not isolated the proxy effect (for example by evaluating the noise-free trimmed statistic and the pooled gap on the same trajectory), so this attribution is a likely explanation, not a measurement. A robust estimate

of the pooled gap would need robust aggregation of per-client rate pairs, weighted by group sizes that are themselves sensitive; we leave this open.

Breakdown and the choice of b. Robustness holds only for $f \leq b$, and beyond it the hiding attack turns the controller against its purpose (Section 7.2). Larger b raises the breakdown point but also the sensitivity: at $K = 5$, $b = 2$ already made the channel unusable on Credit. Small federations therefore cannot tolerate more than one liar with this design, and b must be chosen from an assumption about f that the protocol cannot verify.

Small graphs. With 20–40 validation nodes on the smallest client, the floor $n_{0,\min}$ is 1 or 2 and the noise at $\varepsilon = 8$ exceeds any gap. On German, NBA and Facebook the controller of CLAMP-RT acts on noise, and the improvement of the bias-corrected variant over the uncorrected one there is mostly an effect of the output rule. Larger validation sets, a lower privacy level for the gap channel, or excluding clients with tiny validation sets from the floor would be needed; each has costs we have not evaluated.

Fairness poisoning through updates. The trimmed channel does not touch the update channel. Under fairness poisoning on Pokec-z, CLAMP-RT behaved like clipped aggregation without control, whereas CLAMP with per-round releases held the gap. The slower controller is the likely cause, and the privacy sweep suggests that per-round releases would help on Pokec, but we have not tested this under attack. When the same clients also falsify their gaps, the outcome for $f \leq b$ is about the same as the update attack alone, and for $f > b$ it is worse (Section 7.3). Combining the robust gap channel with a faster controller, or with robust aggregation of updates in secure computation, is an open problem.

Secure computation and noise. The training experiments simulate the two-party protocol in plaintext, and its cost was measured separately in a semi-honest setting with a trusted dealer. A deployment needs a maliciously secure protocol if the servers cannot be trusted to follow it, input validation of the shares, and a discrete noise mechanism over the ring, for which the sensitivity must include fixed-point rounding and the privacy accounting of Theorem 3 must be redone [40, 36]. We do not claim that our continuous-noise analysis transfers unchanged.

Scope of privacy. Theorem 3 protects the attribute of each validation node with respect to the released gaps. It does not protect training-node attributes, which enter the update sum, and it does not make the model differentially private. The value $\varepsilon = 8$ gives a weak worst-case bound; the closed-form attack accuracy of Table 11 (0.57–0.78 at $\varepsilon = 8$, depending on collusion and on how many releases the attacker combines) is a more concrete description.

Guarantees that hold only for the noise-free statistic. Theorem 5 and its consequences concern $\mathrm{TM}_b(\tilde{d})$ before noise. The controller sees the bias-corrected magnitude, which is zero whenever $|\tilde{g}| \leq \sqrt{2}\sigma_d$ (0.065–0.100 on Credit, above the budget), so "liars cannot hide a gap that all honest clients see" does not imply "the controller responds to it". At $f = b$ the condition of consequence (ii) requires every honest client to see the gap.

Scope of the experiments. The attack studies, the privacy sweep and the ablations use three seeds, so only large differences are meaningful there. The attack studies use $K \in \{5, 10\}$, $P = 10$, and for $K = 10$ on three graphs also $P = 1$; none uses more clients. The adaptive attack is myopic and uses one common claim per release; it is not a worst case over all adaptive strategies. The combined attack (poisoned updates and the adaptive gap claim from the same clients) was run only at $K = 10$, $P = 10$ on three graphs. The output-rule fallback rate matters for Theorem 6, which is conditional on no fallback; it was zero in the honest runs, 7 of 126 in the update-attack runs and 35 of 96 in the falsification runs beyond the breakdown point. On German the result is highly variable (0.172 ± 0.189 over ten seeds), so a mean comparison there says little. The attribute probe on the final model was not repeated for CLAMP-RT; the values we cite are for CLAMP, FedAvg and ClipMean. All statistics attacks are omniscient but use simple fixed strategies; the adaptive *opt* attack did no more damage than the fixed hiding attack in our settings. A stronger adaptive adversary may exist, but for $f \leq b$ it cannot place the noise-free estimate outside the interval of Theorem 5, although it can choose its position within that interval in every release. We evaluated one binary attribute and demographic parity only.

9. Conclusion

CLAMP-RT replaces the summed group statistics of the sums-based CLAMP design by one secret-shared scalar per client, sorted, trimmed and privatised by two non-colluding servers. We proved that up to b falsifying

clients cannot move the released estimate outside the range of honest gaps, that the release is attribute-level differentially private with a sensitivity fixed by public validation-set sizes, and a high-probability bound on the released gap of the returned model. In experiments with four fixed attacks and an adaptive one, up to b liars did not move the test gap away from its honest level, where one liar had disabled the sums-based CLAMP. With ten releases that honest level was close to training without control on the Pokec graphs; with per-round releases the controller was active, lowered the gap modestly, and liars with $f \leq b$ again did not change the outcome. Liars that also poisoned their updates added little to the update attack when $f \leq b$ and broke the gap channel when $f > b$. A full release round cost 1.1–10.5 s in a semi-honest two-party implementation, dominated by the norm checks on the updates.

The design has clear costs and limits. With noise on a single trimmed scalar and ten releases, the honest-case gap on Credit and the Pokec graphs was higher than with the sums-based CLAMP; on graphs with a few hundred nodes the noise exceeds the signal; robustness ends at $f = b + 1$, where a hiding attack turns the controller against its purpose; fairness poisoning through updates is not countered; and the training experiments simulate the secure computation with continuous noise. Making the robust channel as accurate as the pooled one, and pairing it with a robust update channel, remain open.

CRediT authorship contribution statement

Quang-Vinh Dang: Conceptualization, Methodology, Formal analysis, Supervision, Writing – original draft, Writing – review & editing. **Ngoc-Son-An Nguyen:** Software, Investigation, Validation, Data curation.

Funding

This research received no funding.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and code availability

All eight datasets are public benchmarks (German, Bail, Credit, Pokec-z, Pokec-n, NBA, Facebook and AMiner-S). The code, the configuration files, the per-run results with per-round logs, and the scripts that generate every table and figure of this paper are available upon publication.

References

[1] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: International Conference on Learning Representations (ICLR), 2017.
URL https://openreview.net/forum?id=SJU4ayYgl

[2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: International Conference on Learning Representations (ICLR), 2018.
URL https://openreview.net/forum?id=rJXMpikCZ

[3] B. McMahan, E. Moore, D. Ramage, S. Hampson, B. Agüera y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Artificial Intelligence and Statistics (AISTATS), Vol. 54, 2017, pp. 1273–1282.
URL https://proceedings.mlr.press/v54/mcmahan17a.html

[4] E. Dai, S. Wang, Say no to the discrimination: Learning fair graph neural networks with limited sensitive attribute information, in: ACM International Conference on Web Search and Data Mining (WSDM), 2021, pp. 680–688. doi:10.1145/3437963.3441752.

[5] C. Agarwal, H. Lakkaraju, M. Zitnik, Towards a unified framework for fair and stable graph representation learning, in: Uncertainty in Artificial Intelligence (UAI), PMLR, 2021, pp. 2114–2124.

[6] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, L. Zhang, Deep learning with differential privacy, in: ACM SIGSAC Conference on Computer and Communications Security (CCS), 2016, pp. 308–318. doi:10.1145/2976749.2978318.

[7] H. B. McMahan, D. Ramage, K. Talwar, L. Zhang, Learning differentially private recurrent language models, in: International Conference on Learning Representations (ICLR), 2018.
URL https://openreview.net/forum?id=BJ0hF1Z0b

[8] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, J. Stainer, Machine learning with adversaries: Byzantine tolerant gradient descent, in: Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 119–129.
URL https://proceedings.neurips.cc/paper/2017/hash/f4b9ec30ad9f68f89b29639786cb62ef-Abstract.html

[9] D. Yin, Y. Chen, K. Ramchandran, P. L. Bartlett, Byzantine-robust distributed learning: Towards optimal statistical rates, in: International Conference on Machine Learning (ICML), Vol. 80, 2018, pp. 5650–5659.
URL https://proceedings.mlr.press/v80/yin18a.html

[10] G. Baruch, M. Baruch, Y. Goldberg, A little is enough: Circumventing defenses for distributed learning, in: Advances in Neural Information Processing Systems (NeurIPS), 2019, pp. 8632–8642.
URL https://proceedings.neurips.cc/paper/2019/hash/ec1c59141046cd1866bbbcdfb6ae31d4-Abstract.html

[11] Y. H. Ezzeldin, S. Yan, C. He, E. Ferrara, A. S. Avestimehr, FairFed: Enabling group fairness in federated learning, in: AAAI Conference on Artificial Intelligence, Vol. 37, 2023, pp. 7494–7502. doi:10.1609/aaai.v37i6.25911.

[12] T. Li, M. Sanjabi, A. Beirami, V. Smith, Fair resource allocation in federated learning, in: International Conference on Learning Representations (ICLR), 2020.
URL https://openreview.net/forum?id=ByexElSYDr

[13] L. Zhang, Z. Han, X. Feng, J. Zhang, Y. Li, C. Chen, FedFACT: A provable framework for controllable group-fairness calibration in federated learning, in: Advances in Neural Information Processing Systems (NeurIPS), 2025.
URL https://arxiv.org/abs/2506.03777

[14] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, K. Seth, Practical secure aggregation for privacy-preserving machine learning, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2017, pp. 1175–1191. doi:10.1145/3133956.3133982.

[15] H. Corrigan-Gibbs, D. Boneh, Prio: Private, robust, and scalable computation of aggregate statistics, in: 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2017, pp. 259–282.

[16] M. Rathee, C. Shen, S. Wagh, R. A. Popa, ELSA: Secure aggregation for federated learning with malicious actors, in: 2023 IEEE Symposium on Security and Privacy (SP), 2023. doi:10.1109/SP46215.2023.10179468.

[17] I. Mironov, Rényi differential privacy, in: IEEE Computer Security Foundations Symposium (CSF), 2017, pp. 263–275. doi:10.1109/CSF.2017.11.

[18] A. Cotter, H. Jiang, M. Gupta, S. Wang, T. Narayan, S. You, K. Sridharan, Optimization with non-differentiable constraints with applications to fairness, recall, churn, and other goals, Journal of Machine Learning Research 20 (172) (2019) 1–59.

[19] B. Knott, S. Venkataraman, A. Hannun, S. Sengupta, M. Ibrahim, L. van der Maaten, CrypTen: Secure multi-party computation meets machine learning, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 34, 2021.

[20] Y. Dong, N. Liu, B. Jalaian, J. Li, EDITS: Modeling and mitigating data bias for graph neural networks, in: The Web Conference (WWW), 2022, pp. 1259–1269. doi:10.1145/3485447.3512173.

[21] C. Yang, J. Liu, Y. Yan, C. Shi, FairSIN: Achieving fairness in graph neural networks through sensitive information neutralization, in: AAAI Conference on Artificial Intelligence, Vol. 38, 2024, pp. 9241–9249. doi:10.1609/aaai.v38i8.28776.

[22] Z. Li, Y. Dong, Q. Liu, J. X. Yu, Rethinking fair graph neural networks from re-balancing, in: ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), 2024, pp. 1736–1745. doi:10.1145/3637528.3671826.

[23] Y. Zhu, J. Li, Y. Bian, Z. Zheng, L. Chen, One fits all: Learning fair graph neural networks for various sensitive attributes, in: ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), 2024. doi:10.1145/3637528.3672029.

[24] L. Corbucci, M. A. Heikkilä, D. Solans Noguero, A. Monreale, N. Kourtellis, PUFFLE: Balancing privacy, utility, and fairness in federated learning, in: European Conference on Artificial Intelligence (ECAI), Vol. 392 of Frontiers in Artificial Intelligence and Applications, 2024, pp. 1639–1647. doi:10.3233/FAIA240671.

[25] X. Ling, J. Fu, K. Wang, H. Li, T. Cheng, Z. Chen, FedFDP: Fairness-aware federated learning with differential privacy, arXiv preprint arXiv:2402.16028 (2024).
URL https://arxiv.org/abs/2402.16028

[26] N. Cui, X. Wang, W. H. Wang, V. Chen, Y. Ning, Equipping federated graph neural networks with structure-aware group fairness, in: IEEE International Conference on Data Mining (ICDM), 2023. doi:10.1109/ICDM58522.2023.00111.

[27] Z. Zhou, S. Yang, F. Zhao, X. Ren, FairGFL: Privacy-preserving fairness-aware federated learning with overlapping subgraphs, IEEE Transactions on Parallel and Distributed Systems 37 (3) (2026). doi:10.1109/TPDS.2025.3649863.

[28] K. Khan, FedGraph-Fair: Federated learning with personalization and fairness via dynamic graphs and distributionally robust optimization, Information Sciences 728 (2026) 122710. doi:10.1016/j.ins.2025.122710.

[29] A.-A. Bendoukha, D. Demirag, N. Kaaniche, A. Boudguiga, R. Sirdey, S. Gambs, Towards privacy-preserving and fairness-aware federated learning framework, Proceedings on Privacy Enhancing Technologies (PoPETs) 2025 (1) (2025) 845–865. doi:10.56553/popets-2025-0044.

[30] Q.-V. Dang, N.-S.-A. Nguyen, FedFairGNN: A privacy-preserving and fairness-aware federated graph neural network for fraud detection, in: Proceedings of IndabaX Nigeria 2026: Building Scalable AI That Works: From Research to Deployment in Resource-Constrained Environments,

Vol. 319 of Proceedings of Machine Learning Research, 2026, pp. 74–86.
URL https://proceedings.mlr.press/v319/dang26a.html

[31] E. M. El Mhamdi, R. Guerraoui, S. Rouault, The hidden vulnerability of distributed learning in byzantium, in: International Conference on Machine Learning (ICML), Vol. 80, 2018, pp. 3521–3530.
URL https://proceedings.mlr.press/v80/mhamdi18a.html

[32] X. Cao, M. Fang, J. Liu, N. Z. Gong, FLTrust: Byzantine-robust federated learning via trust bootstrapping, in: Network and Distributed System Security Symposium (NDSS), 2021. doi:10.14722/ndss.2021.24434.

[33] C. Xie, O. Koyejo, I. Gupta, Fall of empires: Breaking byzantine-tolerant SGD by inner product manipulation, in: Uncertainty in Artificial Intelligence (UAI), Vol. 115, 2019, pp. 261–270.
URL https://proceedings.mlr.press/v115/xie20a.html

[34] H. Lycklama, L. Burkhalter, A. Viand, N. Küchler, A. Hithnawi, RoFL: Robustness of secure federated learning, in: 2023 IEEE Symposium on Security and Privacy (SP), 2023.

[35] J. So, B. Güler, A. S. Avestimehr, Byzantine-resilient secure federated learning, IEEE Journal on Selected Areas in Communications 39 (7) (2021) 2168–2181. doi:10.1109/JSAC.2020.3041404.

[36] P. Kairouz, Z. Liu, T. Steinke, The distributed discrete Gaussian mechanism for federated learning with secure aggregation, in: Proceedings of the 38th International Conference on Machine Learning (ICML), Vol. 139 of Proceedings of Machine Learning Research, 2021, pp. 5201–5212.

[37] C. Dwork, J. Lei, Differential privacy and robust statistics, in: Proceedings of the 41st Annual ACM Symposium on Theory of Computing (STOC), 2009, pp. 371–380.

[38] D. Solans, B. Biggio, C. Castillo, Poisoning attacks on algorithmic fairness, in: Machine Learning and Knowledge Discovery in Databases (ECML PKDD 2020), Vol. 12457 of Lecture Notes in Computer Science, 2020, pp. 162–177.

[39] N. Mehrabi, M. Naveed, F. Morstatter, A. Galstyan, Exacerbating algorithmic bias through fairness attacks, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 35, 2021, pp. 8930–8938.

[40] C. L. Canonne, G. Kamath, T. Steinke, The discrete Gaussian for differential privacy, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020, pp. 15676–15688.

[41] J. McAuley, J. Leskovec, Learning to discover social circles in ego networks, in: Advances in Neural Information Processing Systems (NIPS), Vol. 25, 2012.

[42] Y. Dong, Z. Lei, Z. Zheng, S. Wang, J. Ma, A. J. Huang, C. Chen, J. Li, PyGDebias: A Python library for debiasing in graph learning, in: Companion Proceedings of the ACM Web Conference 2024 (WWW '24 Companion), 2024, pp. 1019–1022. doi:10.1145/3589335.3651239.